

Zero-Knowledge Proofs of Generalized Regular Expression Matching for Anonymized Email Verification

Anonymous Author(s)

Abstract

Digital communication increasingly underpins identity, financial transactions, and regulatory compliance. In many settings, possession of a DKIM-signed email serves as evidence of account control, transaction confirmation, or institutional affiliation. Yet demonstrating such properties typically requires revealing the full email or relying on centralized intermediaries, introducing privacy risks and additional trust assumptions. A framework called ZK Email addresses this limitation by applying zero-knowledge proofs (ZKPs) to email verification, enabling publicly verifiable proofs of authenticity while preserving message confidentiality. However, its existing implementations struggle to support complex, real-world messages due to the inefficiency of regular-expression verification over structured formats and rich alphabets.

We address this limitation with a new ZKP system for generalized regular expression (regex) matching that replaces in-circuit automaton simulation with efficient path verification over ϵ -free NFAs, yielding prover complexity linear in the input length and independent of the regex complexity. This approach enables practical validation of expressive standard structures required for full DKIM-signed email verification. By integrating our constructions into ZK Email, we obtain an enhanced system that combines (i) DKIM signature verification, (ii) an arbitrary-length SHA-256 circuit with partial precomputation for `rsa-sha256` under RFC 6376, and (iii) a general-purpose regex primitive enforcing structural constraints over email headers and body. We formalize the associated zero-knowledge relations and analyze their security under realistic adversary models. We implement the system (over 40,000 lines of code released under MIT) in Circom and Noir, targeting `groth16` and `ultra_honk` backends, and evaluate it in both client-side and zkVM (SP1) deployment settings. Experimental results on a commodity hardware demonstrate substantial efficiency improvements over prior DFA-based approaches, achieving 2–6 $\times$ in proving time, while supporting a significantly richer class of regex languages.

Keywords

Zero-knowledge proofs, regular expression, finite automata, Email provenance

1 Introduction

Digital systems increasingly depend on structured user-submitted data, such as emails, for identity verification, tax and residency declarations, and KYC/AML compliance. In most deployments, however, such messages lack portable cryptographic guarantees that

can be verified independently of the original service provider. Authenticity is therefore established through ad hoc checks or reliance on trusted intermediaries. This limitation is amplified by generative tools capable of producing synthetic yet highly realistic emails and other formally structured communications [23].

Email infrastructure already provides a widely deployed provenance mechanism through DomainKeys Identified Mail (DKIM) [1, 8]. A DKIM-signed email contains a verifiable signature issued by the sending domain over selected header fields and the message body. This signature can serve as a root of authenticity. In many emerging applications, however, email-derived claims must be verified in decentralized or publicly verifiable settings, such as smart contracts (e.g., user-friendly cryptocurrency wallet management) or shared execution environments (e.g., Cloud virtual machines) where no single party is trusted to perform the validation. In these contexts, naively transmitting the full content of a specific email and verifying its DKIM signature directly is impractical. The message size can lead to high data transmission costs, signature verification might be computationally expensive in constrained environments, and in most of the scenarios, applications often require selective disclosure of specific fields rather than full message publication.

An immediate mitigation is to rely on centralized attestations, whereby external providers validate emails or account ownership and issue confirmations. Although straightforward to deploy, this model introduces additional trust assumptions, privacy risks, and operational constraints. Verifiers often require direct access to parts of user’s email account and may retain sensitive correspondence or become single points of failure. Enabling self-contained cryptographic proofs of origin and structural validity for email offers a more robust and independently verifiable alternative [4].

Recent work such as [4] has demonstrated that privacy-enhancing primitives such as zero-knowledge proofs (ZKPs) can integrate DKIM verification to enable selective and privacy-preserving email authentication. The objective is to allow a user to prove properties of a received email—such as sender authenticity, structural validity of headers, or the presence of specific fields—without disclosing the full message. Proof generation is performed client-side, while verification requires only a succinct proof and limited public inputs. Such a system enables trustless verification of DKIM-signed communications for applications including privacy-preserving identity proofs (e.g., account control or organizational membership), decentralized financial bridges and email-based wallets via on-chain verification of payment confirmations, and reusable anonymous KYC through selective disclosure [12]. It further can be employed to support pseudonymous legal and financial attestations, such as transaction receipts, tax filings, or proof of residence.

However, the main technical challenge of this approach is efficient enforcement of highly expressive, structured textual constraints required for DKIM compliance within the ZK domain. Email headers and bodies are defined by regular-language grammars and

This work is licensed under the Creative Commons Attribution 4.0 International License. To view a copy of this license visit <https://creativecommons.org/licenses/by/4.0/> or send a letter to Creative Commons, PO Box 1866, Mountain View, CA 94042, USA.

Proceedings on Privacy Enhancing Technologies YYYY(X), 1–25

© YYYY Copyright held by the owner/author(s).

<https://doi.org/XXXXXXXX.XXXXXXX>



are commonly validated using complex regular expression (regex). Recent efforts to prove regex membership in ZK [4, 10] either support restricted automata patterns, impose bounds on message length, or incur prover complexity that scales with the size of the input. These limitations render them impractical for full DKIM support in realistic deployments.

We propose a general-purpose zero-knowledge regular-expression primitive (ZK-Regex) that proves membership of a private string in a language defined by an ϵ -free nondeterministic finite automaton (NFA), with prover complexity independent of the size of the automaton and linear in the matching path length. Our construction separates matching from verification: the prover computes a valid accepting run (valid path in NFA) off-circuit, and the ZK circuit verifies correctness of the claimed state transitions and acceptance. The resulting constraint system scales linearly with the length of the accepting NFA path and is independent of the overall message length (entire email size). We integrate this primitive with DKIM signature verification to obtain end-to-end authenticity-preserving proofs for real-world emails. Concretely, our framework combines: (i) zero-knowledge verification of DKIM signatures; (ii) an arbitrary-length SHA-256 circuit with support for partial precomputation (rsa-sha256 under RFC 6376 [28]); and (iii) a general regex verification primitive compiled from ϵ -NFA.

We implement the proposed framework as a complete system comprising all zero-knowledge circuits required for DKIM signature verification, arbitrary-length SHA-256 hashing with partial precomputation, and general-purpose regex verification. The circuits are realized in two domain-specific ZK languages, Circom [13] and Noir [31], targeting the groth16 [21] and ultra_honk [11] proving backends, respectively. The codebase consists of over 40,000 lines of ZK circuit design and integration code. At the application layer, the system includes smart contracts for on-chain verification and SDKs supporting custom circuit compilation, client-side proof generation, and selective disclosure workflows. All components are released as open-source under the MIT license.

We deploy and evaluate the system in realistic settings on commodity hardware (MacBook Pro M4 Pro) and formally reduce the security of our constructions to well-established cryptographic assumptions. Fine-grained benchmarks demonstrate that the proposed ZK-Regex method achieves a 2–6 $\times$ improvement in proving time over prior state-of-the-art approaches, with end-to-end client-side ZK email proving times as low as 3.8 s and verification times of approximately 200 ms. We further evaluate deployability, including setup requirements and on-chain verification complexity, indicating low maintenance overhead and practical deployment costs.

The main contributions of this work are as follows.

- **ZKPs for DKIM verification.** We design and implement an end-to-end ZK construction for verifying DKIM-signed emails that is fully compatible with existing standards, enabling proofs of sender authenticity and structured message properties without revealing full email content.
- **Efficient dynamic hashing of large messages in ZK.** We introduce an arbitrary-length SHA-256 circuit with partial precomputation that reduces constraint overhead while remaining fully compliant with RFC 6376.

- **Input-size independent ZK regex verification.** To the best of our knowledge, we are the first to develop a general ZK regex verification primitive whose circuit complexity scales linearly in the path length and is independent of the input string size, enabling efficient validation of large and expressive structured document formats.
- **Implementation and evaluation of concrete efficiency.** We provide a production-ready implementation in Circom and Noir, evaluate performance across groth16 and ultra_honk for client-side proving as well as a zkVM (SP1) [29] server-side proving. We release all of the circuits, contracts, and SDKs under MIT license for practical deployment.

The remainder of the paper is organized as follows. Section 2 reviews the necessary background. Section 3 presents the protocol design. Section 4 defines the system and adversary models and analyzes the security of the top-level NIZK relations, including unforgeability, anonymity, and linkability. Section 5 describes implementation details and optimizations. Section 6 reports experimental results. Section 7 discusses related work.

2 Background

2.1 Finite Automata

Definition 2.1. (Deterministic Finite Automaton (DFA)) We formally define a DFA as a 5-tuple

$$M = \langle Q, \Sigma, \delta, q_0, F \rangle,$$

where

- Q is a finite set of states, indexed $0, \dots, |Q| - 1$;
- Σ is a finite input alphabet (e.g. ASCII bytes);
- $\delta : Q \times \Sigma \rightarrow Q$ is the total transition function;
- $q_0 \in Q$ is the start state;
- $F \subseteq Q$ is the set of accepting states.

A *transition table* is a tabular representation of δ :

$$\text{table}[i][j] = k \quad \text{iff} \quad \delta(q_i, \sigma_j) = q_k,$$

where $\{\sigma_j\}$ enumerates Σ . Because δ is total and functional, each row has exactly $|\Sigma|$ entries.

Example. Consider the regular expression ab^*c over $\Sigma = \{a, b, c\}$. Let us enumerate the states as $Q = \{q_0, q_1, q_2, q_{\text{dead}}\}$ with q_0 start, q_2 accepting, and q_{dead} a sink state. Figure 1 visualizes this DFA using [5]. The transition matrix $T \in Q^{|Q| \times |\Sigma|}$ is:

	a	b	c
q_0	q_1	q_{dead}	q_{dead}
q_1	q_{dead}	q_1	q_2
q_2	q_{dead}	q_{dead}	q_{dead}
q_{dead}	q_{dead}	q_{dead}	q_{dead}

Definition 2.2. (Nondeterministic Finite Automaton (NFA)) We formally define an NFA machine as a similar 5-tuple

$$M = \langle Q, \Sigma, \delta, q_0, F \rangle,$$

with the same Q, Σ, q_0, F as above, but where

$$\delta : Q \times \Sigma \rightarrow \mathcal{P}(Q).$$

Here, $\mathcal{P}(Q)$ denotes the *power set* of Q . δ is a *transition relation*, not necessarily functional. From state $q \in Q$ and symbol $\sigma \in \Sigma$, there may be zero, one, or multiple next states $\{q' \mid (q, \sigma, q') \in \delta\}$.

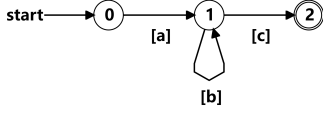


Figure 1: DFA for the Regex ab^*c , generated by [5].

Definition 2.3. (ϵ -NFA) An ϵ -NFA generalizes an NFA by allowing transitions on the empty string:

$$N_\epsilon = \langle Q, \Sigma, \delta_\epsilon, q_0, F \rangle,$$

where

$$\delta_\epsilon : Q \times (\Sigma \cup \{\epsilon\}) \rightarrow \mathcal{P}(Q).$$

An ϵ -move (q, ϵ, q') consumes no input; the extended transition relation δ_ϵ^* is the reflexive transitive closure including such moves.

Definition 2.4. (Acceptance conditions) Let $w = a_1 a_2 \dots a_n$ be a string over the alphabet Σ . The automaton M accepts w if we could see a perfect, continuing path of state transitions: $q_0 \xrightarrow{a_1} q_i \xrightarrow{a_2} \dots q_k$, where q_k is one of the possible acceptance states. In other words and more formally, there exists a sequence of states $r_0, r_1, \dots, r_n$ with the following properties:

- (1) $r_0 = q_0$,
- (2) $\forall i \in [n] : r_{i+1} = \delta(r_i, a_{i+1})$,
- (3) $r_n \in F$.

The first condition specifies that the machine starts in the initial state q_0 . The second condition requires that, for each input symbol a_i , the machine transitions to the next state according to δ . The final condition requires that, after reading the entire string, the automaton halts in an accepting state. If these conditions are met, M accepts w ; otherwise, it rejects w . The set of strings that M accepts is called the *language recognized by M* , denoted $L(M)$.

Equivalently, acceptance can be defined using the extended transition function $\delta^* : Q \times \Sigma^* \rightarrow \mathcal{P}(Q)$:

$$\delta^*(r, \epsilon) = \{r\},$$

$$\forall (x \in \Sigma^*, a \in \Sigma) : \delta^*(r, xa) = \bigcup_{s \in \delta^*(r, x)} \delta(s, a).$$

Here, $\delta^*(r, x)$ is the set of all states reachable from r by consuming x . A string w is accepted if

$$\delta^*(q_0, w) \cap F \neq \emptyset.$$

Implementation Notes. For zero-knowledge circuit embedding, these automata can be represented using:

- a dense transition matrix for DFAs (constant-time lookup);
- per-state sparse lists or range compression for NFAs;
- flattened arrays with offsets for compact storage of Δ or Λ_ϵ .

Such representations allow circuit constraints to enforce valid state transitions succinctly while supporting cryptographic commitments to the automaton structure.

2.2 Verification of Email-Originated Data

Public-key cryptography provides a well-established method for verifying the authenticity of data: a party can confirm that data originated from a specific entity by verifying a digital signature created with the private key of the entity. Although this principle underpins many secure systems, it has seen limited deployment in user-facing provenance workflows.

However, the electronic mail system offers an underutilized vector for cryptographic attestations. Most email traffic today includes domain-authenticated digital signatures via the DKIM standard. DKIM signs a canonicalized hash of selected email headers and message body using the domain's private key. The core form of the signature can be summarized as follows.

```
rsa_sign(sha256(from: . . . , to: . . . , subject: . . . ,
               <body_hash>, . . . ), private_key)
```

Since approximately 2017, major email providers have consistently signed outbound messages using DKIM. As a result, a received email from `someperson@mit.edu` to `you@yourdomain.com` can be cryptographically validated by fetching `mit.edu`'s public key from its DNS TXT records and verifying the DKIM signature.

Because signed headers include a hash of the body, one can also prove the content of the email. For example, the receipt of a Twitter verification email can be used to attest to ownership of a Twitter account, using only the DKIM header and relevant email content.

2.3 Non-Interactive Zero-Knowledge Arguments of Knowledge

We follow the original definition from [21] to formally define Non-Interactive Zero-Knowledge Arguments of Knowledge.

Definition 2.5. Let $\mathcal{R}$ be a relation generator that, given a security parameter λ in unary, outputs a binary relation R that is decidable in polynomial time. For any pair $(\phi, w) \in R$, we refer to ϕ as the *statement* and w as the *witness*. Let $\mathcal{R}_\lambda$ denote the set of relations that $\mathcal{R}$ may output when given input 1^λ . Now, we define a publicly verifiable non-interactive argument system for $\mathcal{R}$ as a quadruple of probabilistic polynomial algorithms $\Pi = (\text{Setup}, \text{Prv}, \text{Vfy}, \text{Sim})$ such that:

- $(\sigma, \tau) \leftarrow \text{Setup}(R)$: produces the CRS σ and a trapdoor τ for the relation R .
- $\pi \leftarrow \text{Prv}(R, \sigma, \phi, w)$: given that $(\phi, w) \in R$, the algorithm returns an argument π .
- $b \leftarrow \text{Vfy}(R, \sigma, \phi, \pi)$: using the same statement ϕ and CRS σ used in generating π , the algorithm Vfy returns $b \in \mathbb{Z}_2$: 0 (reject) or 1 (accept).
- $\pi \leftarrow \text{Sim}(R, \tau, \phi)$: Given the trapdoor τ and statement ϕ , the simulator can return the argument π .

Definition 2.6. According to [21], we can label Π as a “*perfect non-interactive zero-knowledge argument of knowledge*” if it satisfies following properties: **Perfect Completeness**: An honest prover with valid witness should convince an honest verifier. Formally:

$$\Pr \left[\text{Vfy}(R, \sigma, \phi, \pi) = 1 \mid \begin{array}{l} (\sigma, \tau) \leftarrow \text{Setup}(R) \\ (\phi, w) \in R \\ \pi \leftarrow \text{Prv}(R, \sigma, \phi, w) \end{array} \right] = 1$$

Definition 2.7. Computational Soundness: The Π is sound if it is not possible to prove a false statement. Let L_R denote the language consisting of all the statements such as ϕ that have matching witness in R . Therefore, for all probabilistic polynomial-time (PPT) adversaries $\mathcal{A}$:

$$\Pr \left[\begin{array}{c} \text{Vfy}(R, \sigma, \phi, \pi) = 1, \\ \phi \notin L_R \end{array} \middle| \begin{array}{c} (R, z) \leftarrow \mathcal{R}(1^\lambda) \\ (\sigma, \tau) \leftarrow \text{Setup}(R) \\ (\phi, \pi) \leftarrow \mathcal{A}(R, z, \sigma) \end{array} \right] = \text{negl}(\lambda)$$

Computational Knowledge Soundness: To call Π an “*argument of knowledge*”, we require existence of a PPT extractor $\chi_{\mathcal{A}}$ for all PPT adversaries $\mathcal{A}$ that can compute the witness whenever the adversary manages to provide a valid argument:

$$\Pr \left[\begin{array}{c} \text{Vfy}(R, \sigma, \phi, \pi) = 1, \\ (\phi, w) \notin R \end{array} \middle| \begin{array}{c} (R, z) \leftarrow \mathcal{R}(1^\lambda) \\ (\sigma, \tau) \leftarrow \text{Setup}(R) \\ ((\phi, \pi), w) \leftarrow (\mathcal{A} \parallel \chi_{\mathcal{A}})(R, z, \sigma) \end{array} \right] = \text{negl}(\lambda)$$

Definition 2.8. Perfect Zero-knowledge: The argument does not leak anything beyond the truth of statement. We say Π is perfect zero-knowledge if for all adversaries $\mathcal{A}$ with access to z as an auxiliary side information derived from relation generator:

$$\Pr \left[\begin{array}{c} \mathcal{A}(R, z, \sigma, \tau, \pi) = 1 \\ (\phi, w) \in R \end{array} \middle| \begin{array}{c} (\sigma, \tau) \leftarrow \text{Setup}(R) \\ \pi \leftarrow \text{Prv}(R, \sigma, \phi, w) \end{array} \right] = \Pr \left[\begin{array}{c} \mathcal{A}(R, z, \sigma, \tau, \pi) = 1 \\ (\phi, w) \in R \end{array} \middle| \begin{array}{c} (\sigma, \tau) \leftarrow \text{Setup}(R) \\ \pi \leftarrow \text{Sim}(R, \tau, \phi) \end{array} \right]$$

3 ZKPs for DKIM-Signed Emails

Throughout this section, we assume a setting where the prover aims to demonstrate the receipt of an email from a known domain (e.g., google.com), prove that its message body satisfies a given regular expression, and selectively reveal specific portions of the message. We construct a NIZK relation $\mathcal{R}_{\text{Email}}$ that verifies the authenticity and structure of an email protected under DKIM. The prover demonstrates that jointly the cryptographic signature and the syntactic validity of the email through an embedded regex verifier. The prover generates a ZK proof π_σ for the **statement** $\phi = (pk_{\text{srvr}}, pp_{\text{RSA}}, m^*, \text{rgx}_{\text{ptrn}})$, where pp_{RSA} denotes the public RSA parameters used for DKIM signature¹, pk_{srvr} the mail server’s public key², rgx_{ptrn} a predefined masking regex tailored to the expected format of m , producing a regex-consistent view of the message m^* , and **witness** $w = (\sigma_{\text{DKIM}}, \ell_m, m)$, consisting of a valid DKIM signature σ_{DKIM} on the received message m , and its length ℓ_m . Formally, the NIZK proof π_{Email} proves membership in the relation $\mathcal{R}_{\text{Email}}$ defined as follows:

$$\mathcal{R}_{\text{Email}} := \{ (\phi = (pk_{\text{srvr}}, pp_{\text{RSA}}, m^*, \text{rgx}_{\text{ptrn}}), w = (\sigma_{\text{DKIM}}, \ell_m, m)) : \begin{array}{l} \text{sigVerify}(H(m), pk_{\text{srvr}}, \sigma_{\text{DKIM}}, pp_{\text{RSA}}) = 1 \wedge \\ \text{isValidDKIM}(m) = 1 \wedge m^* = \text{match}(m, \text{rgx}_{\text{ptrn}}) \end{array} \} \quad (1)$$

The public input vector x consists of: $x = (pk_{\text{srvr}}, pp_{\text{RSA}}, m^*, \text{rgx}_{\text{ptrn}})$. The witness w provided by the prover is $w = (\sigma_{\text{DKIM}}, \ell_m, m)$. The circuit enforces that $(x, w) \in \mathcal{R}_{\text{Email}}$ if and only if all following hold:

¹Corresponding to RSA modulus and public exponent, published in the DNS TXT records as part of the DKIM public key.

²The sender’s address follows the standard username@domain format, where the domain is represented by the mail server’s public key pk_{srvr} , whose corresponding private key sk_{srvr} signs the message.

- (1) **Cryptographic correctness:** The SHA-256 hash of the message m and the RSA signature verification both succeed, ensuring the authenticity of the claimed sender: $\text{sigVerify}(H(m), pk_{\text{srvr}}, \sigma_{\text{DKIM}}, pp_{\text{RSA}}) = 1$.
- (2) **Syntactic validity:** $\text{isValidDKIM}(m) = 1$, confirming that the message satisfies the canonicalization and formatting constraints defined by the DKIM standard.
- (3) **Selector consistency:** $m^* = \text{match}(m, \text{rgx}_{\text{ptrn}})$, ensuring that the revealed masked message is consistent with both m and the specified masking rule.

Finally, the verifier (e.g. a smart contract) checks that the on-chain RSA public key pk_{srvr} corresponds to the legitimate DKIM key published by the sender domain in DNS.

3.1 Arbitrary-Length Hash Verification

A central challenge in supporting general email verification is handling variable-length message bodies within a single verifier circuit. To this end, we extend the circomlib[2] implementation of SHA-256 to support arbitrary-length inputs up to a configurable bound.

To mitigate the prohibitive constraint cost of hashing entire messages on-chain, we employ a modular precomputation strategy inspired by Merkle-Damgård and sponge constructions[14, 16]. Let the email body be split into blocks $m_1, \dots, m_t$. The hash is computed iteratively as $h_i = H(h_{i-1}, m_i)$. The prover computes h_{i-1} off-circuit for all but the final few blocks and provides h_{i-1} as a public input. The circuit then verifies that applying SHA-256 to the final sequence of blocks produces the claimed digest. Formally, the circuit proves knowledge of $m_k, \dots, m_t$ such that:

$$H^{(t-k+1)}(h_{k-1}, m_k, \dots, m_t) = h_t.$$

This approach maintains completeness and soundness while reducing constraint overhead from linear in the total message size to linear in the number of tail blocks.

3.2 Regex Matching in Zero Knowledge

Correct parsing of email headers requires more than substring detection. The verifier must ensure that specific fields (e.g., From, To, Subject) appear in valid positions and are delimited by canonical separators such as `\r\n`. Naïve substring checks are insufficient, as an adversary may embed misleading header-like strings within other fields (e.g., inside the message body). To enforce proper parsing semantics, we require zero-knowledge verification of regular-language constraints over private text. This will give us the ability to prove the presence of structured patterns in private text without revealing the text itself. This primitive enables privacy-preserving proofs of account ownership, transaction confirmation, regulatory notice receipt, and secure recovery workflows.

3.2.1 Direct DFA Encoding (Baseline Approach). A straightforward method for ZK regex verification is to compile the regular expression into a DFA and embed its transition function directly into the arithmetic circuit. Prior work [4] follows this approach. Each transition is encoded as arithmetic constraints, and the circuit simulates the DFA step-by-step over the input string.

This design supports selective disclosure by decomposing the entire regex into smaller sub-patterns (“*part regexes*”) and decide on visibility of each independently, e.g. as *public/private* and therefore,

exposing/hiding the corresponding matched substrings. Figure 2 illustrates this approach. The DFA structure is embedded directly into the arithmetic circuit, enabling in-circuit evaluation of transitions. Listing 1 provides an example of dividing the input regex into multiple parts with varying configurations. We provide more details of DFA-to-Circuit translation technique used in [4] in Appendix A.

However, directly encoding the DFA transition table introduces structural limitations. The circuit size scales with both the number of DFA states $|Q|$ and the input length $|w|$, since each character requires evaluation against the full transition logic. For complex expressions, this results in large constraint systems and high proving costs. Furthermore, DFA-based constructions restrict support for richer regex features, such as non-greedy quantifiers ($*?$ or $+?$), alternations ($|$), and require careful manual composition when combining multiple sub-patterns to preserve semantic equivalence [4].

```

1 { "parts": [
2   { "is_public": false,
3     "regex_def": "email:" },
4   { "is_public": true,
5     "regex_def": "[a-z]+@[a-z]+.com" },
6   { "is_public": false,
7     "regex_def": "." } ] }

```

Listing 1: Example JSON structure with regex definitions.

3.2.2 Off-Circuit Matching, In-Circuit Verification. In proposed construction, we avoid simulating the full automaton inside the circuit. Instead, we separate matching from verification. Figure 3 illustrates the architectural differences between the baseline and proposed approaches. The proposed method consists of three main steps, as follows.

1) Regex Compilation: Input regex is compiled into an NFA and a corresponding verification circuit. NFA structure is derived using a Thompson-style construction [35] via `regex_automata` library [3], supporting ϵ -transitions for native capture group boundaries.

2) Off-Circuit Matching: The prover runs NFA on the input text to compute a match and produces a traversal path: a sequence of transitions (q_i, c_i, q_{i+1}) , i.e. (current_state, input_byte, next_state) that should be accepted the NFA machine:

$$((p_0, a_1, p_1), (p_1, a_2, p_2), \dots, (p_{k-1}, a_k, p_k)),$$

with $p_0 = q'_0$, $p_k \in F'$, and substring $y = a_1 a_2 \dots a_k$ of x .

3) In-Circuit Verification: The circuit takes the claimed match and path as private inputs and verifies the exact acceptance criteria formalized in Definition 2.4 as follows:

- the path starts in a valid initial state,
- each claimed transition is valid according to NFA rules,
- the state sequence is contiguous, and
- the final state is accepting.

Formally, the circuit verifies the following NIZK relation:

$$\mathcal{R}_{\text{regex}} := \{((C_{[c_1, \dots, c_k]}), (w_{[a_1, \dots, a_k]}), P_{[p_0, \dots, p_k]}) : p_0 = q_0 \wedge p_k \in F \wedge \forall i \in [k] : (p_{i-1}, a_i, p_i) \in \delta \wedge c_i = \text{Cap}(a_i, p_{i-1})\}.$$

Here, the only public input is the vector $C = [c_1, \dots, c_k]$, representing the claimed capture-group activity for each transition in

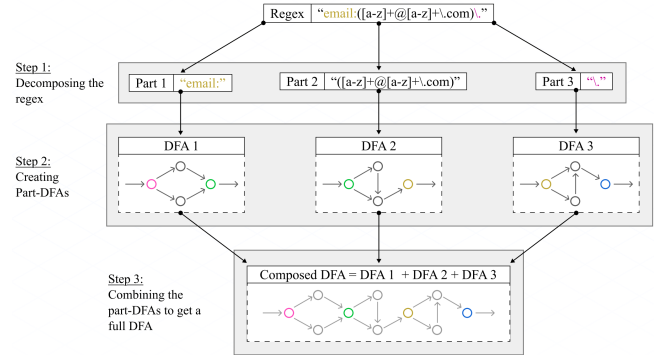


Figure 2: [Baseline approach] Decomposition of a regex into part-DFAs with varying visibility.

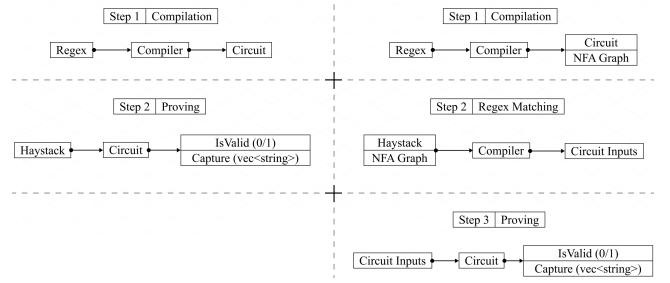


Figure 3: ZK Regex pipeline in baseline (left side) vs. the proposed approach (right side).

the matched path. The private witness consists of the full regex-compatible string $w = [a_1, \dots, a_k]$ and a valid state path $P = [p_0, \dots, p_k]$ such that the sequence of transitions follows the automaton and terminates in an accepting state ($p_k \in F$). The function $\text{Cap}(a_i, p_{i-1})$ extracts the capture information associated with symbol a_i at state p_{i-1} , as defined by the regex semantics.

4 Security Analysis

4.1 System and Threat Model

We adopt a standard probabilistic polynomial-time (PPT) adversary model, consistent with prior work [12]. The adversary $\mathcal{A}$ may corrupt and fully control a prover attempting to convince an honest verifier of a false claim about a target email. Consistent with real-world deployments, we assume that the sending and receiving mail servers are honest and uncompromised; in particular, their signing keys and authentication mechanisms remain secure. When controlling an entity, $\mathcal{A}$ may:

- Observe, intercept, modify, delay, or inject arbitrary messages on behalf of the corrupted actor.
- Attempt to forge zero-knowledge proofs.
- Mount substitution attacks by generating proofs that bind to a different account than the one that legitimately sent or received the email.
- Produce multiple conflicting proofs for the same email or replay previously valid proofs in a different context.

- Adaptively choose inputs, queries, and proof statements in order to maximize the probability that an honest verifier accepts without knowledge of a valid witness.
- Attempt to extract or infer sensitive information from proofs generated by honest provers.

We assume that $\mathcal{A}$ cannot:

- Break the existential unforgeability of the signature scheme underlying the DKIM standard.
- Find collisions in the hash function H .
- Violate the soundness or zero-knowledge properties of the underlying zkSNARK system, such as Groth16 [21] or UltraHonk [11].

Furthermore, we assume a realistic deployment setting consistent with the current email ecosystem. In particular, DKIM public keys are obtained from DNS records and are assumed to correspond to the legitimate domain unless otherwise stated [28]. We do not assume universal deployment of DNSSEC, and therefore key authenticity may rely on auxiliary trust mechanisms, such as governance-based validation or hard-coded key registries, depending on the deployment model [30]. We also assume that mail servers operate according to the standard DKIM specification, including correct canonicalization and signature generation. Finally, we acknowledge that real-world deployments introduce additional trust and governance considerations, including DNS key rotation, censorship resistance, decentralized key validation, and limitations arising from mail server visibility [24, 32, 37]. For conciseness, we omit a detailed treatment of these operational assumptions in the main body of the paper. A comprehensive discussion of the system model, deployment considerations, and trust assumptions is provided in [Appendix B](#).

4.2 Correctness

We require our protocol to be correct. As this property is rather straightforward, we omit a formal definition. We say if computed proof of an honest user successfully gets verified by any honest verifier, the scheme is correct. This property holds because the underlying cryptographic schemes are correct, the signature scheme σ fulfill correctness, and because the NIZK is complete. Furthermore, for $\mathcal{R}_{\text{regex}}$, if a regex match exists, a valid proof can always be generated, as the ε -elimination process [22] preserves the matching semantics of the original regex.

4.3 Unforgeability

We establish the unforgeability of our construction by analyzing the forgery probability of the top level NIZK relation $\mathcal{R}_{\text{Email}}$. Here, we provide only a sketch of the formal analysis used to prove the theorems. Full proofs appear in [Appendix C](#).

THEOREM 1. *Assuming underlying signature scheme SIG unforgeable, H is collision-resistant, and NIZK is knowledge-sound, our construction $\mathcal{R}_{\text{Email}}$ is unforgeable w.r.t. $\text{GameUnforge}_{\mathcal{A}}^{\text{Email}}$ in Figure 4. It holds that:*

$$\Pr[\text{GameUnforge}_{\mathcal{A}}^{\text{Email}}(\lambda) = 1] \leq \text{negl}(\lambda)$$

SKETCH. We model the mail server as a standard signing oracle with the additional constraint that it signs only messages that satisfy

the DKIM canonicalization and formatting rules (Line 1 of $\mathcal{O}^{\text{Mail}}$). Since the mail server is assumed to be honest and its signing key uncompromised, any valid signature σ_m must have been generated on a correctly formed message m . By the unforgeability of the underlying signature scheme, an adversary cannot produce a valid σ_m for a message that was not signed by the server as they do not have access to the private key sk_{srvr} .

An alternative attack strategy would be to modify a previously signed message m while preserving its hash value h_m . This would require finding a collision in the hash function H used in the DKIM.Verify&Sign procedure. However, by the collision resistance of H , it is computationally infeasible to find two distinct messages $m' \neq m$ such that $H(m') = H(m)$.

Finally, the soundness of the underlying NIZK proof system guarantees that no adversary can produce an accepting proof without knowledge of a valid witness for the specified relation ($\mathcal{R}_{\text{Email}}$). $\square$

THEOREM 2. *Assuming the underlying NIZK argument system is knowledge-sound, the construction $\mathcal{R}_{\text{regex}}$ is unforgeable.*

SKETCH. The proof is straightforward and follows by reduction to the knowledge soundness of the underlying NIZK argument system. The relation $\mathcal{R}_{\text{regex}}$ verifies a witness encoding a valid ε -NFA execution trace over the input string: (i) starts from an initial state; (ii) each transition corresponds to a valid edge of the NFA encoded in the circuit; and (iii) terminates in an accepting state. Thus, any accepting proof implies the existence of a valid NFA path consistent with the encoded transition relation and the input. A forgery would therefore produce an accepting proof without a corresponding valid execution trace, contradicting the knowledge soundness of the underlying NIZK. $\square$

The unforgeability in Theorem 2 mitigates following attacks:

- *Invalid transition injection:* Each claimed transition is checked against the immutable transition relation embedded in the circuit, preventing the introduction of non-existent edges.
- *Input substitution:* Transitions are verified with respect to the provided byte sequence, binding the proof to a specific input and preventing reuse across different messages.
- *Capture group manipulation:* Capture boundaries are enforced by the NFA structure and validated at each transition, preventing tampering with selectively revealed substrings.

4.4 Anonymity

We require that the proposed scheme provide anonymity in the following sense: given two valid claims produced by two different users, if the publicly disclosed message component m^* is identical in both cases, then no PPT adversary can distinguish between the proofs. In our model, the mail server (i.e., the holder of the signing key pk_{srvr}) is assumed to be honest and non-colluding with the adversary. As in the soundness analysis, we provide only a proof sketch in here, while the complete proofs are given in [Appendix C](#).

THEOREM 3. *Assuming underlying NIZK system is zero-knowledge, our construction $\mathcal{R}_{\text{Email}}$ fulfills anonymity w.r.t. $\text{GameAnon}_{\mathcal{A}}^{\text{Email}}$ in Figure 5. It holds that:*

$$\left| \Pr[\text{GameAnon}_{\mathcal{A}}^{\text{Email}}(\lambda) = 1] - \frac{1}{2} \right| = \text{negl}(\lambda)$$

```

GameUnforge $_{\mathcal{A}}^{\text{Email}}(\lambda)$ :
1 :  $\text{rgx}_{\text{ptrn}} \leftarrow \text{Setup}(\lambda)$ ;
2 :  $sk_{\text{srvr}} \xleftarrow{\$} \{0, 1\}^\lambda$ ;  $pk_{\text{srvr}} \leftarrow \text{PubKey}(sk)$ ;
3 :  $S \leftarrow \emptyset$ ;  $S_H \leftarrow \emptyset$ ;
4 :  $\{m_i\}_{i \in [n]} \xleftarrow{\$} \mathcal{A}(pk_{\text{srvr}}, \text{rgx}_{\text{ptrn}})$ ;
5 :  $(\pi'_{\text{Email}}, m^*) \xleftarrow{\$} \mathcal{A}^{O^{\text{Mail}}}(pk_{\text{srvr}}, \text{rgx}_{\text{ptrn}}, \{m_i\}_{i \in [n]})$ ;
6 : return  $\mathcal{R}_{\text{Email}}.\text{Vfy}_{\pi}(\pi'_{\text{Email}}, pk_{\text{srvr}}, m^*, \text{rgx}_{\text{ptrn}}) = 1 \wedge$ 
7 :  $\exists m' \text{ s.t. } m' \notin S \wedge m^* = \text{match}(m', \text{rgx}_{\text{ptrn}})$ ;

 $O^{\text{Mail}}(m)$ :
1 :  $\sigma_m \leftarrow \text{DKIM}.\text{Verify\&Sign}(H, m, sk_{\text{srvr}})$ ;
2 :  $S \leftarrow S \cup \{m\}$ ;
3 : return  $\sigma_m$ ;

 $H(m) : O^{\text{Rand}}(m)$ :
1 : if  $\exists h, \text{ s.t. } (m, h) \in S_H$  : return  $h$ ;
2 : else:  $h \xleftarrow{\$} \{0, 1\}^\lambda$ ;  $S_H \leftarrow S_H \cup \{(m, h)\}$ ; return  $h$ ;

```

Figure 4: Game-based definition of unforgeability for $\mathcal{R}_{\text{Email}}$.

SKETCH. Recall that we need to prove that two valid proofs of two different original messages m_0 and m_1 , both satisfying the regex pattern rgx_{ptrn} , are computationally indistinguishable. Let m^* denote a masked message consistent with rgx_{ptrn} . Consider the procedure UnMask (Line 4 of Figure 5), which, given m^* and rgx_{ptrn} , produces two distinct messages $m_0 \neq m_1$ such that both conform to the pattern rgx_{ptrn} . By construction, both messages yield the same public statement in the anonymity game. By the zero-knowledge property of the underlying NIZK proof system, the proof π_{m_b} reveals nothing beyond the associated public statement. Since the public statements pk_{srvr} and m^* are identical whether the witness corresponds to m_0 or m_1 , the distributions of the resulting proofs π_{m_0} and π_{m_1} are computationally indistinguishable. Thus, no PPT adversary can win the game with non-negligible advantage, implying the anonymity of the scheme. $\square$

4.5 Linkability

Finally, we require that our scheme be linkable, implying that an adversary cannot create two different regex-consistent views of the message $m_0^* \neq m_1^*$, given the exactly same original message m and the same rgx_{ptrn} . We provide only a proof sketch in here, while the complete proofs are given in Appendix C.

THEOREM 4. *Assuming underlying NIZK system is knowledge-sound, H is collision-resistant, and SIG is unforgeable, our construction $\mathcal{R}_{\text{Email}}$ fulfills linkability w.r.t. $\text{GameLink}_{\mathcal{A}}^{\text{Email}}$ in Figure 6. It holds that:*

$$\Pr[\text{GameLink}_{\mathcal{A}}^{\text{Email}}(\lambda) = 1] \leq \text{negl}(\lambda)$$

SKETCH. We show that no PPT adversary, given a single DKIM signature σ_m on a message m , can produce two verifying proofs

```

GameAnon $_{\mathcal{A}}^{\text{Email}}(\lambda)$ :
1 :  $\text{rgx}_{\text{ptrn}} \leftarrow \text{Setup}(\lambda)$ ;
2 :  $sk_{\text{srvr}} \xleftarrow{\$} \{0, 1\}^\lambda$ ;  $pk_{\text{srvr}} \leftarrow \text{PubKey}(sk)$ ;
3 :  $m^* \leftarrow \mathcal{A}$ ;
4 :  $\{\langle m_0, \ell_{m_0} \rangle, \langle m_1, \ell_{m_1} \rangle\} \xleftarrow{\$} \text{UnMask}(m^*, \text{rgx}_{\text{ptrn}})$ ;
5 :  $b \xleftarrow{\$} \{0, 1\}$ ;
6 :  $\sigma_{m_b} \leftarrow \text{DKIM}.\text{Verify\&Sign}(H, m_b, sk_{\text{srvr}})$ ;
7 :  $\pi_{m_b} \leftarrow \mathcal{R}_{\text{Email}}.\text{Prv}(\sigma_{m_b}, \ell_{m_b}, m_b)$ ;
8 :  $b^* \leftarrow \mathcal{A}(\pi_{m_b}, m^*, pk_{\text{srvr}})$ ;
9 : return  $b^* == b$ ;

```

Figure 5: Game-based definition of Anonymity for $\mathcal{R}_{\text{Email}}$.

```

GameLink $_{\mathcal{A}}^{\text{Email}}(\lambda)$ :
1 :  $\text{rgx}_{\text{ptrn}} \leftarrow \text{Setup}(\lambda)$ ;
2 :  $sk_{\text{srvr}} \xleftarrow{\$} \{0, 1\}^\lambda$ ;  $pk_{\text{srvr}} \leftarrow \text{PubKey}(sk)$ ;
3 :  $m \leftarrow \mathcal{A}()$ ;
4 :  $\sigma_m \leftarrow \text{DKIM}.\text{Verify\&Sign}(H, m, sk_{\text{srvr}})$ ;
5 :  $(\pi'_0, \pi'_1, m_0^*, m_1^*) \xleftarrow{\$} \mathcal{A}(pk_{\text{srvr}}, m, \sigma_m, \text{rgx}_{\text{ptrn}})$ ;
6 : return  $(m_0^* \neq m_1^*) \wedge$ 
7 :  $(\forall i \in \{0, 1\} : \text{Vfy}_{\pi}(\pi'_i, pk_{\text{srvr}}, m_i^*, \text{rgx}_{\text{ptrn}}) = 1)$ ;

```

Figure 6: Game-based definition of linkability for $\mathcal{R}_{\text{Email}}$.

corresponding to two distinct masked messages $m_0^* \neq m_1^*$ derived from the same m .

By the soundness of the NIZK proof system, any accepting proof for relation $\mathcal{R}_{\text{Email}}$ implies the existence of a valid witness. In particular: (i) the masked message m^* was correctly derived from an underlying message m , (ii) the message satisfies the required DKIM regex constraints, and (iii) there exists a valid signature σ_m under the mail server's public key pk_{srvr} verifying the DKIM structure and binding to m .

Suppose an adversary outputs two accepting proofs with $m_0^* \neq m_1^*$. By the soundness of the NIZK proof system, both proofs must correspond to valid witnesses. Since the adversary is given only a single signature σ_m on m , generating a second valid but distinct masked message can only occur in one of the following ways: 1) The adversary finds a distinct message $m' \neq m$ such that $H(m') = H(m)$, thereby reusing the same signature, contradicting its collision resistance. Or 2) the adversary produces a new valid signature $\sigma'_{m'}$ under pk_{srvr} for some $m' \neq m$, violating the unforgeability of the underlying signature scheme SIG . $\square$

5 Implementation Details

Encoding Strategy for Noir. Noir employs a sparse array data structure to efficiently encode and verify transition rules. A sparse array stores only non-zero entries together with their indices, making it well-suited for NFAs, where most state-to-state transitions

are undefined. Each valid transition (s, b, s') is uniquely mapped to an index key via a *Random Linear Combination (RLC)*:

$$\text{key} = s + b \cdot 257 + s' \cdot 257^2.$$

This encoding provides three key properties:

- **Unique positioning:** Each component occupies a distinct digit position in base-257 arithmetic.
- **Collision-freedom:** Since $257 > 255$ (the maximum byte value), no two distinct transitions produce the same key.
- **Constant-time lookup:** The circuit can check a claimed transition by computing its key and querying the sparse array at that index.

For example, the transition $(s = 5, b = 65, s' = 12)$ yields

$$\text{key} = 5 + 65 \cdot 257 + 12 \cdot 257^2 = 807,378.$$

Each sparse array entry stores a validity flag and any associated capture-group markers, enabling $O(1)$ lookup time rather than iterating through all possible transitions. This strategy substantially reduces circuit complexity for regexes with large transition sets.

Performance Characteristics. The total number of transitions is proportional to the regex complexity (e.g., use of character classes, alternations, and repetitions). The performance implications differ across circuit DSLs:

- **Circom:** Here the circuit size scales as $O(\text{match_length} \times \text{total_transitions})$, since each input byte requires iterating over all possible transitions.
- **Noir:** Circuit size scales as $O(\text{match_length} \times \text{lookup_cost})$, where the lookup cost is effectively constant due to the direct key-based access of the sparse array.

For regexes with large transition sets, Noir’s sparse-array encoding yields significantly more efficient circuits compared to the exhaustive iteration model used in Circom.

5.1 Proposed zk-Regex Capabilities

The proposed zk-Regex architecture introduces several significant improvements over the *baseline* [4] in terms of expressiveness, security, performance, and modularity, by decoupling regex matching from in-circuit verification.

Expanded Regex Support. The primary advantage of proposed scheme is its ability to support a much wider range of regular expression syntax. By moving the complex matching logic off-circuit, proposed zk-Regex can handle any pattern that the `regex_automata` library can compile into a NFA. This enables robust support for features that were difficult or impossible to implement in baseline [4] approach’s direct-to-circuit model, including:

- Non-greedy quantifiers (e.g., `*, ?+, +?`);
- Complex alternations (`|`) and nested grouping;
- Unicode character classes and properties (e.g., `\p{L}`).

We provide detailed comparison of different regex patterns supported by each method in Appendix F (Check Table 15 for different patterns). This approach also eliminates the need for the manual “decomposed regex” strategy from baseline. Users can now specify a single, complete regex pattern—simplifying circuit construction and reducing potential implementation errors.

Unsupported Features. zk-Regex inherits the deliberate design constraints of `regex_automata` [3] library, which omits certain features to ensure linear-time matching performance. As a result, the proposed zk-regex does not support:

- Lookarounds (e.g., `(?=...)`, `(?!...)`, `(?<=...)`, `(?<!=...)`);
- Backreferences (e.g., `\1`, `\k<name>`);
- Other complex PCRE [7] constructs such as atomic groups.

Regex patterns containing these features will fail to compile.

Performance and Scalability. The proposed method architecture fundamentally changes the scaling behavior of zk-Regex:

- **Decoupled complexity:** Circuit size is no longer tied to regex complexity, but primarily to the maximum match length and the number of capture groups.
- **Targeted proving:** The proving cost scales with the length of the matched substring, not the full haystack.
- **Efficient resource use:** CPU-intensive matching occurs off-circuit, while the circuit handles only lightweight path verification.

Although direct quantitative comparison with the *baseline* approach is non-trivial due to architectural differences, replacing in-circuit automaton simulation with explicit path verification yields substantially improved scalability for complex regular expressions. We report comprehensive benchmark results in Appendix F.

Proving-System-Agnostic Core. The Rust-based compiler underlying the proposed zk-Regex framework performs regex parsing, NFA construction, and ϵ -elimination. The design is proving-system agnostic and produces a standardized intermediate representation, NFAGraph, which can be compiled into backend-specific formats for systems such as Circom and Noir. The same implementation can also be executed directly within zkVM environments, including SP1. Additional details on supported proving backends and their evaluation are provided in Appendix D.

Simplified and Accurate Capture Groups. In *baseline* [4], capture groups were implemented through manual regex decomposition. In our method, they are handled natively by the NFA, and therefore, capture groups are intrinsic to the NFA produced by `regex_automata` as the epsilon elimination preserves their start and end boundaries. This ensures semantically correct and predictable behavior consistent with standard regex engines [22].

5.2 Epsilon-Free NFA Generation

To make NFA verification efficient within a ZK circuit, the automaton must be transformed into an equivalent one that does not use ϵ -transitions. Here we describe the transformation process.

From Regex to an Initial NFA. The process begins by constructing a NFA from the input regex string. Proposed zk-Regex employs Rust’s `regex_automata` library [3] for this task. This approach provides two key benefits:

- **Optimized Construction:** The library implements a variant of Thompson’s NFA construction, producing NFAs that are structurally aligned with the regex and typically more compact than equivalent DFAs.
- **Native Capture Groups:** The generated NFAs include special ϵ -transitions (via the `State::Capture` variant) to mark

capture-group boundaries. This yields a unified automaton that encodes capture-group semantics directly, avoiding the manual and fragile composition of part-DFAs required in *baseline*.

The output of this stage is an NFA that correctly represents the regex but still contains ϵ -transitions, which are incompatible with direct circuit verification.

The Problem with ϵ -transition. An ϵ -transition allows an NFA to move between states without consuming an input byte. While such transitions are standard for modeling alternation ($|$), repetition ($*$, $+$), and grouping, they pose challenges for circuit verification:

- (1) **Invisible Steps:** Modeling state changes that do not consume input bytes introduces conditional logic and variable steps in circuit, significantly inflating constraint counts.
- (2) **Verification of Gapped Paths:** A traversal path containing ϵ -transitions has “gaps” in the byte sequence. The circuit would need to prove that these gaps correspond exactly to valid ϵ -paths—an expensive and complex process.

Thus, all ϵ -transitions should be removed before circuit generation.

The ϵ -elimination Algorithm. The algorithm converts the initial NFA into an equivalent ϵ -free NFA while preserving both matching semantics and capture-group boundaries. The transformation, illustrated in Figure 7, operates under the assumption that every meaningful capture group consumes at least one byte. The algorithm proceeds as follows:

- (1) **Compute ϵ Closures:** For each state S , compute its ϵ -closure—the set of all states reachable from S via only ϵ -transitions. During this traversal, record any encountered capture-group events (start/end markers) and accept states.
- (2) **Rewire Transitions:** For every state S and each state R in its ϵ -closure, if R has an outgoing byte transition $R \xrightarrow{\text{byte}} T$, add a direct transition $S \xrightarrow{\text{byte}} T$. This bypasses ϵ -only paths from S to R while maintaining equivalent behavior.
- (3) **Preserve Capture-Group Semantics:** When adding a direct transition $S \xrightarrow{\text{byte}} T$:
 - **Start Captures:** Attach any start markers found along the ϵ -path from S to R to this new transition, indicating that the capture begins with consumption of this byte.
 - **End Captures:** Attach any end markers found in the ϵ -closure of T , ensuring that the capture ends when the byte leading to T is consumed.

This two-sided transfer correctly delimits captured substrings in the resulting automaton.

- (4) **Update Start and Accept States:** If the ϵ -closure of a state includes an accept state, that state itself becomes an accept state. Similarly, start states are updated to include all states reachable via ϵ -transitions from the original start state.
- (5) **Prune Unreachable States:** Perform a reachability analysis from the updated start states and remove any states that can no longer be reached.

The resulting NFA is **behaviorally equivalent** to the original but entirely ϵ -free [22]. Each state transition now consumes a byte, and all capture-group boundaries are precisely preserved. This makes

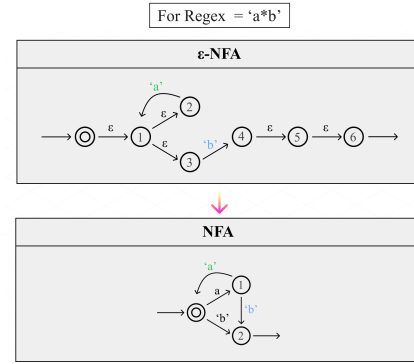


Figure 7: Caption

the structure suitable for direct encoding as arithmetic constraints within a zero-knowledge circuit.

5.3 Mapping NFAs to Arithmetic Circuits

With an ϵ -free NFA constructed, the next step is to encode its semantics into an arithmetic-circuit-compatible form. This section describes how the NFA is represented within the circuit and how the off-circuit matching logic interacts with in-circuit verification.

Flattening the NFA into a Transition Set. An ϵ -free NFA is a graph-based structure. To embed it within a circuit, we first *flatten* its transition rules into a structured set of tuples representing all valid state changes shown in Figure 8. The process produces three data components: 1) set of valid start states, 2) set of valid accept states, and 3) complete set of byte-level transitions, where each transition is a tuple (source_state, byte_range, destination_state) optionally annotated with capture-group metadata.

This transition set forms an immutable ruleset embedded within the zero-knowledge circuit. The method for enforcing these rules depends on the capabilities of the target proving system — in particular, whether it natively supports operations such as associative lookups, unconstrained witness computation, and efficient range checks. When such features are available, the circuit can verify transitions with significantly fewer constraints.

Circum (R1CS). R1CS represents computations as flat collections of rank-1 quadratic equations over field elements [21, 36]; higher-level control flow, memory, and data structures (e.g., hash maps, dynamically-sized arrays) must be encoded manually into these constraints rather than being supported natively. As a result, the circuit must iterate through the entire transition set for each input byte, arithmetically checking that the claimed state transition matches at least one valid tuple. Each check instantiates comparison templates (IsEqual, LessThan) that decompose into multiple R1CS constraints, producing a cost of $O(T \times M)$ constraints where T is the number of NFA transitions and M is the match length.

Noir (UltraHonk). UltraHonk’s arithmetization supports lookup gates and unconstrained witness computation, enabling a fundamentally different encoding [11]. All valid transitions are packed into a sparse array keyed by a composite (current_state, byte, next_state) value. At proving time, a binary search is performed

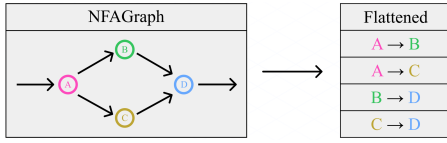


Figure 8: Flattening Process

in an unconstrained function — executing outside the constraint system at zero gate cost — and the result is verified with two native range-check assertions at a fixed cost of approximately 14.5 gates per lookup, independent of the number of NFA transitions. This reduces the per-byte verification cost from $O(T)$ to $O(1)$.

6 Experimental Results

6.1 Setup and Benchmark Configurations

All benchmarks were executed on a Macbook Pro M4 (14core) with 48 GB of RAM. Table 3 in Appendix D.1 summarizes the exact hardware and software configuration.

Methodology. For each configuration, the circuit is compiled once and proofs are generated multiple times. All of the runs are evaluated multiple times and we report the median across runs to reduce sensitivity to system-level variance. We also exclude the first proof generation run to avoid warm-up effects. Appendix E provides a detailed statistical analysis of benchmarking quality and consistency³.

6.2 ZK Circuit Complexity

Constraint and Gate Scaling. We analyze prover complexity by measuring the number of constraints as the total email input size (header + body), increases. We evaluate circuit complexity scaling with input length across both proving systems. Detailed system-specific results are provided in Appendix D (Figures 17 and 18). Figure 9 shows both systems exhibit near-identical linear scaling when normalized. The slopes are similar despite different absolute units (R1CS constraints vs. UltraHonk gates). Exact values are reported in Appendix D, Table 4. Both Circom and Noir achieve $R^2 \approx 0.99$ with respect to total input size. The per-byte cost is approximately 545 constraints/B in Circom and 87 gates/B in Noir.

Feature Flag Overhead. To have a better understanding of the individual parameters of impacting circuit complexity, We isolate the cost of individual features by varying a single flag in Circom or substituting a single circuit template in Noir against a common baseline. In Circom, the baseline is FEAT-BASE (640h/768b, all flags disabled, 960,202 constraints). In Noir, the baseline is FEAT-BASE (512h/1024b, verify_email, 192,406 gates). Figures 10 and 11 report feature overhead as the percentage change relative to FEAT-BASE (960,202 constraints for Circom and 192,406 gates for Noir)⁴.

³In Circom, Groth16 requires a per-circuit trusted setup (zkey), which is performed once per configuration and is excluded from the reported proving-time measurements.

⁴In Circom, [†]FEAT-SOFT is confounded: maxBodyLength is set to 1408 bytes, compared to 768 bytes in all other configurations. This increases the constraint count independently of the removeSoftLineBreaks flag and inflates the apparent overhead. In contrast, Noir’s FEAT-SOFT uses the same maxBodyLength=1024 as the baseline, providing a clean measurement of the soft line break removal cost (+13.3%).

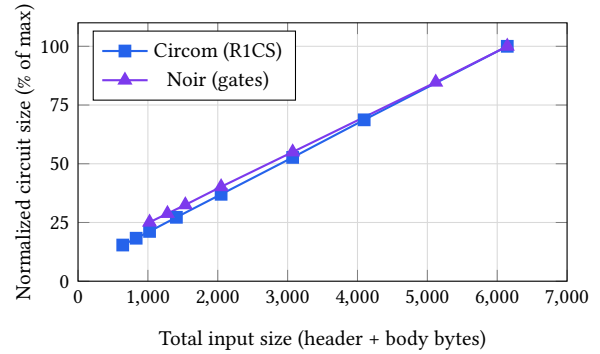


Figure 9: Circom vs. Noir: normalized circuit size.

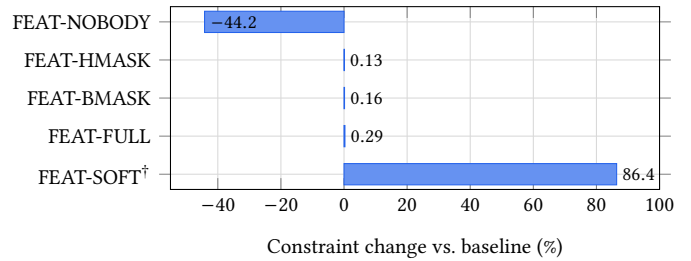


Figure 10: Circom: feature flag impact on constraints.

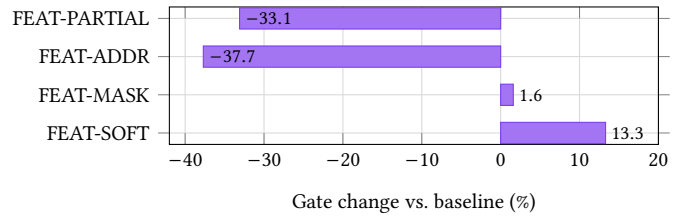


Figure 11: Noir: feature flag impact on constraints.

We provide a detailed analysis of the impact of individual feature flags on circuit complexity in Appendix D, Figure 19 and Table 5. Overall, header and body masking introduce negligible overhead (less than 2%) in both systems. Skipping or partially computing the body hash yields the largest reduction in constraints and gates (33–44%).

RSA Key Size Impact. We evaluate the impact of the RSA public key size on circuit complexity in Appendix D, Table 7. Doubling the RSA key size from 1024 to 2048 bits adds only ~10% gate overhead. RSA verification is a relatively small fraction of the total circuit (SHA-256 hashing dominates).

SHA-256 Precomputation. We evaluate the impact of the SHA-256 partial precomputation technique described in Section 3.2. A complete analysis is provided in Appendix D (Figure 21, Table 8, and Figure 22). In Noir, 75% precomputation results in **33% fewer gates** and **39% faster proving**. In Circom, constraint reduction requires manually decreasing maxBodyLength, yielding –18% at

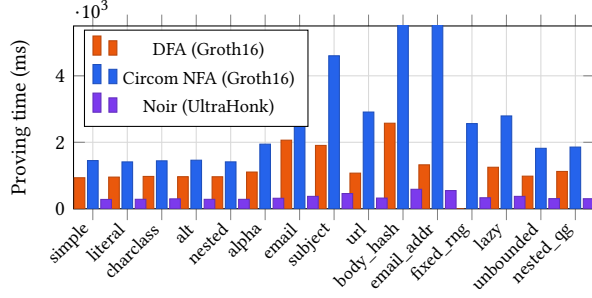
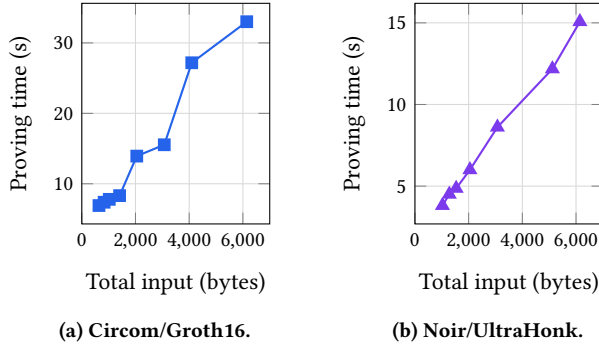


Figure 12: ZK regex Proving time at 64-byte input.



(a) Circom/Groth16.

(b) Noir/UltraHonk.

Figure 13: Proving time vs. total input size for both systems.

50% and -25% at 80%. Due to the fixed overhead of the partial-hash template in Noir, precomputation becomes net beneficial only beyond approximately 40% off-circuit hashing.

6.3 Prover Complexity

Figure 13 shows that proving time scales approximately linearly with input size in both systems. Circom/Groth16 exhibits a steeper slope and visible steps (e.g., between 3,000 and 4,000 bytes), corresponding to a ptau power-of-two boundary. For a better comparison, Figure 14 reports the prover performance of Noir relative (at the same input size) to Circom. Proving is consistently 1.8–2.3× faster in Noir, while witness generation is 3.2–6.3× faster. We provide a more detailed analysis in Appendix D.3.

To evaluate the efficiency of the proposed ZK regex construction, we compare it against the DFA-based approach of [4] across multiple representative patterns, detailed in Table 15. Figure 12 reports proving times for inputs of 64 bytes. Our Noir-based implementation achieves a 2–6× speedup over [4] under these settings. Moreover, the Noir backend consistently outperforms our Circom-based implementation by a factor of 5–15×. Additional experimental details are provided in Appendix F.5.

6.4 Verification and Communication Complexity

Groth16 verification remains effectively constant at approximately ~188 ms, independent of circuit size. Although this behavior is

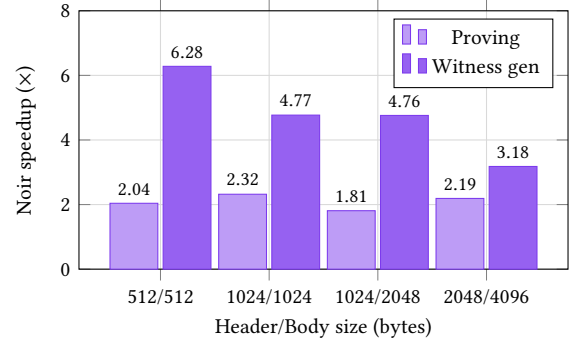


Figure 14: Noir speedup ratios over Circom.

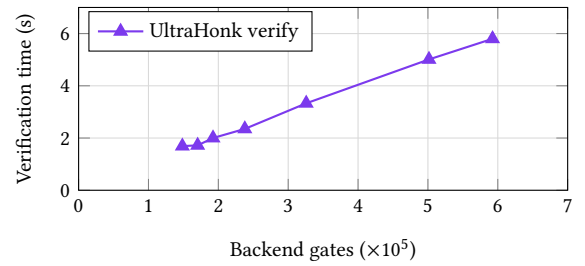


Figure 15: Noir/UltraHonk verification time.

Table 1: Groth16 vs. UltraHonk system-level tradeoffs.

Criterion	Groth16 (Circom)	UltraHonk (Noir)
Proof size	805 B	14,080 B
Verification time	188 ms (constant)	1.7 s–5.8 s (scales)
Proving time	6.9 s–33.0 s	3.8 s–15.1 s
Witness generation	1.3 s–4.8 s	250 ms–1.5 s
Compilation	8 s–68 s	1.7 s–5.4 s
Trusted setup	Required (per-circuit)	Not required
Max circuit size	$\sim 4.19 \times 10^6$ constraints	No hard limit
On-chain suitability	Excellent	Limited (large proofs)

expected, for completeness we benchmark detailed verifier performance under different configurations (see Appendix D.4, Table 9). In contrast, as shown in Figure 15, Noir/UltraHonk verification time scales linearly with circuit size, increasing from 1.7 s at 148 k gates to 5.8 s at 592 k gates. This differs from Groth16’s constant $O(1)$ verification complexity.

6.5 End-to-End Performance

We evaluate both systems with respect to deployment-related overhead, including compilation time and, for Groth16, the additional cost of deriving a new zkey for each circuit as part of the final trusted setup phase (see Appendix D.5).

We compare Circom and Noir at identical input sizes (header/body bytes). Four configurations share matching dimensions: 512/512, 1024/1024, 1024/2048, and 2048/4096. Figure 16 reports the total end-to-end time, including witness generation, proving, and verification, for both backends. Table 1 summarizes the benchmark results and highlights the main tradeoffs of each implementation.

Table 2: Overview of state-of-the-art zk-regex approaches.

System	Automata	Complexity	Proof System	Trusted Setup	Proof Size	Features
Reef [10]	SAFA	$O(\alpha \times \log N)$	Nova (IVC)	No	Sublinear	Full PCRE
Zombie [38]	Thompson NFA	$O(D \times Q)$	Various	Varies	Varies	No lookahead
zkreg [33]	AC-DFA	$O(D + Q)$	Groth16	Yes	~660 B/proof	Fixed strings only
zk-regex (baseline) [4]	Min-DFA	$O(D \times Q_{DFA})$	Groth16	Yes (per-circuit)	805 B	Restricted
Ours (Circom)	ϵ -free NFA	$O(m \times T)$	Groth16	Yes (per-circuit)	805 B	Most regex
Ours (Noir)	ϵ -free NFA	$O(m \times \text{lookup})$	UltraHonk	No	14,080 B	Most regex

Where: m = match length, T = transitions, α = non-skip steps, N = document + automaton size.

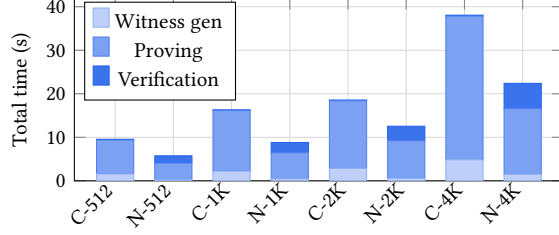


Figure 16: End-to-end performance. C = Circom, N = Noir.

7 Related Work

Verifiable Parsing and zk-Regex. Reef [10] is an exploration of zk-Regex. However, it is not directly applicable in the DKIM setting. Reef assumes that the document D is committed using a multilinear polynomial commitment that enables efficient random access during proof generation, whereas DKIM requires a SHA-256 commitment of the email body. Bridging these two commitments would require an additional zero-knowledge proof of commitment equivalence, introducing significant overhead. Moreover, Reef’s recursive design depends on Nova [27], which is not practical for computations involving non-IVC-friendly operations such as RSA, even with optimizations like Cyclefold [26]. Its skipping automata also optimize for patterns of the form $.+$ but not bounded negations such as $[\wedge;]+$, which dominate DKIM regexes. While we expect future recursive systems to improve the efficiency of such designs, the current Reef implementation does not outperform our Noir-based [11, 31] circuits for end-to-end email verification in practice. Table 2 summarizes state-of-the-art zk-regex systems across automata models, asymptotic complexity, proof assumptions, and supported features.

Privacy-Preserving Identity and Token-Based Proofs. The European Digital Identity Wallet (EUDIW) Architecture and Reference Framework emphasizes user control, data minimization, and interoperability in a standardized wallet-based architecture [19]. These goals have motivated research into using zero-knowledge proofs for selective disclosure and unlinkability in digital identity systems [9]. zkLogin [12] operationalizes this approach by using ZKPs to prove control over OpenID Connect (OIDC) [34] identity tokens (JSON Web Tokens (JWT)) and derive unlinkable blockchain signing keys via Tagged Witness Signatures. While zkLogin demonstrates the feasibility of proving statements about identity provider-issued tokens, it remains inherently tied to third-party authentication providers and blockchain-specific account bootstrapping.

A recent study analyzes different vulnerabilities in zkLogin [15]. Some of these weaknesses can be mitigated by using our constructions. Most importantly, we avoid ad-hoc substring parsing and claim shadowing by enforcing strict regular-language constraints through $(\mathcal{R}_{\text{regex}})$, where parsing semantics are encoded in a fixed automaton and each transition is verified in-circuit. This prevents duplicate-field ambiguity, character smuggling, and malformed-structure attacks similar to those identified in JWT parsing.

Internet Trust Infrastructure. FreeAuth [20] addresses privacy and security limitations in conventional email ownership verification, which typically relies on sending verification emails that expose full addresses and introduce interception, spam, and tracking risks. The protocol leverages existing SMTP/IMAP/POP3 authentication flows (via SASL) to prove email ownership without revealing the underlying address or requiring challenge emails. It employs TLS-oracle techniques to disclose only the authentication outcome and combines two-party computation (2PC) with zero-knowledge proofs to produce privacy-preserving statements about email identifiers. While FreeAuth demonstrates that ownership of an email account can be proven without direct disclosure, it does not leverage cryptographic artifacts already embedded in emails—such as DKIM signatures—and does not support selective proofs over structured message content or metadata.

NOPE [18] proposes a server authentication mechanism that reduces reliance on the traditional Certificate Authority (CA) ecosystem by requiring domains to prove, via a succinct zkSNARK, that their TLS public key is bound to a valid DNSSEC chain of trust. NOPE demonstrates how succinct proofs can strengthen web PKI without disrupting existing TLS infrastructure. Some of the similar work in this domain are [17, 30]. While NOPE strengthens domain-level authentication in the transport layer, our scheme enables privacy-preserving provenance proofs at the application layer over real-world communications.

Decentralized Storage and Provenance. Hello [25] studies abuse in InterPlanetary File System (IPFS)-based content distribution, demonstrating practical attacks enabled by anonymous pinning and gateway caching, and advocates stronger accountability mechanisms such as stricter KYC, content moderation, and ZK-based identity tools. This line of work underscores the importance of provenance in decentralized storage systems. Our method complements this direction by providing cryptographic provenance for message-origin data itself, for example, enabling an email that seeds an IPFS object to carry a verifiable, privacy-preserving attestation of origin when stored or redistributed through IPFS.

Acknowledgment

The authors used generative AI-based tools to revise the text, improve flow and correct any typos, grammatical errors, and awkward phrasing.

References

- [1] 2011. *Domainkeys identified mail (DKIM) signatures*. Technical Report.
- [2] 2024. CircomLib. <https://github.com/iden3/circomlib/>. Accessed: 2026-02-07.
- [3] 2024. regex automata library in Rust. https://docs.rs/regex-automata/latest/regex_automata/. Accessed: 2026-02-17.
- [4] 2024. ZK Email Baseline. <https://zk.email/blog/zkemail#user-content-fnref-3>. Accessed: 2026-02-17.
- [5] 2025. Regex DFA Graph Visualizer. https://zkregex.com/min_dfa. Accessed: 2025-09-31.
- [6] 2026. How does DNSSEC work? <https://www.cloudflare.com/learning/dns/dnssec/how-dnssec-works/>. Accessed: 2026-02-20.
- [7] 2026. Perl-compatible regular expressions (PCRE). <https://www.pcre.org/original/doc/html/pcresyntax.html>. Accessed: 12 February 2026.
- [8] 2026. What is a DNS DKIM record? <https://www.cloudflare.com/learning/dns/dns-records/dns-dkim-record/>. Accessed: 2026-02-20.
- [9] Iván Abellán Álvarez, Pol Hölzmer, and Johannes Sedlmeir. 2025. Privacy evaluation of the European Digital Identity Wallet's architecture and reference framework. *Computers & Security* (2025), 104707.
- [10] Sebastian Angel, Eleftherios Ioannidis, Elizabeth Margolin, Srinath Setty, and Jess Woods. 2024. Reef: Fast succinct {Non-Interactive} {Zero-Knowledge} regex proofs. In *33rd USENIX Security Symposium (USENIX Security 24)*. 3801–3818.
- [11] AztecProtocol. 2025. Barretenberg: An optimized elliptic-curve library and PLONK SNARK prover. <https://github.com/AztecProtocol/aztec-packagesbarretenberginaztec-packages>.
- [12] Foteini Baldimtsi, Konstantinos Kryptos Chalkias, Yan Ji, Jonas Lindström, Deepak Maram, Ben Riva, Arnab Roy, Mahdi Sedaghat, and Joy Wang. 2024. zklogin: Privacy-preserving blockchain authentication with existing credentials. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*. 3182–3196.
- [13] Marta Bellés-Muñoz, Miguel Isabel, Jose Luis Muñoz-Tapia, Albert Rubio, and Jordi Baylina. 2022. Circom: A Circuit Description Language for Building Zero-knowledge Applications. *IEEE Transactions on Dependable and Secure Computing* (2022), 1–18. <https://doi.org/10.1109/TDSC.2022.3232813>
- [14] Guido Bertoni, Joan Daemen, Michaël Peeters, and Gilles Van Assche. 2007. Sponge functions. In *ECRYPT hash workshop*, Vol. 2007.
- [15] Sofia Celi, Hamed Haddadi, and Kyle Den Hartog. 2026. Analysis and Vulnerabilities in zkLogin. *Cryptology ePrint Archive* (2026).
- [16] Jean-Sébastien Coron, Yevgeniy Dodis, Cécile Malinaud, and Prashant Puniya. 2005. Merkle-Damgård revisited: How to construct a hash function. In *Annual International Cryptology Conference*. Springer, 430–448.
- [17] Antoine Delignat-Lavaud, Cédric Fournet, Markulf Kohlweiss, and Bryan Parno. 2016. Cinderella: Turning shabby X.509 certificates into elegant anonymous credentials with the magic of verifiable computation. In *2016 IEEE Symposium on Security and Privacy (SP)*. IEEE, 235–254.
- [18] Zachary DeStefano, Jeff J Ma, Joseph Bonneau, and Michael Walfish. 2024. NOPE: Strengthening domain authentication with succinct proofs. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles*. 673–692.
- [19] European Commission. 2024. European Digital Identity Wallet Architecture and Reference Framework. Technical architecture and specifications for the European Digital Identity Wallet.
- [20] Yijia Fang, Bingyu Li, Jiale Xiao, Bo Qin, Zhijintong Zhang, and Qianhong Wu. 2024. FreeAuth: Privacy-Preserving Email Ownership Authentication with Verification-Email-Free. In *2024 Annual Computer Security Applications Conference (ACSAC)*. IEEE, 336–352.
- [21] Jens Groth. 2016. On the Size of Pairing-Based Non-interactive Arguments. In *Advances in Cryptology – EUROCRYPT 2016*, Marc Fischlin and Jean-Sébastien Coron (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 305–326.
- [22] John E. Hopcroft, Rajeev Motwani, and Jeffrey D. Ullman. 2001. *Introduction to Automata Theory, Languages, and Computation* (2 ed.). Addison-Wesley.
- [23] IBM X-Force Red. 2023. AI vs. human deceit: Unravelling the new age of phishing tactics. Available at: <https://www.ibm.com/think/x-force/ai-vs-human-deceit-unravelling-new-age-phishing-tactics/> (accessed 25 February 2026).
- [24] et al. Ishtiaq. 2026. A Longitudinal and Qualitative Study of Email and DNS Security. In *Proceedings of the ACM Conference on Computer and Communications Security*.
- [25] Christos Karapapas, Iakovos Pittaras, George C Polyzos, and Constantinos Pat-sakis. 2025. Hello, won't you tell me your name?: Investigating Anonymity Abuse in IPFS. In *International Conference on Availability, Reliability and Security*. Springer, 187–204.
- [26] Abhiram Kothapalli and Srinath Setty. 2023. CycleFold: Folding-scheme-based recursive arguments over a cycle of elliptic curves. *Cryptology ePrint Archive*, Report 2023/1192. <https://eprint.iacr.org/2023/1192>
- [27] Abhiram Kothapalli, Srinath Setty, and Ioanna Tzialla. 2022. Nova: Recursive Zero-Knowledge Arguments from Folding Schemes. 359–388. https://doi.org/10.1007/978-3-031-15985-5_13
- [28] Murray Kucherawy, Dave Crocker, and Tony Hansen. 2011. DomainKeys Identified Mail (DKIM) Signatures. RFC 6376. <https://doi.org/10.17487/RFC6376>
- [29] Succinct Labs. 2026. SP1. <https://github.com/succinctlabs/sp1>. Accessed: Mar 1, 2026.
- [30] Netlas.io. 2025. An Expert's View on DNSSEC: Pros, Cons, and When to Implement. https://netlas.io/blog/what_is_dnssec/. Accessed: 27 February 2026.
- [31] Noir Development Team. 2025. Noir: A Domain-Specific Language for Zero-Knowledge Proofs. [urlhttps://noir-lang.org/](https://noir-lang.org/). Version v1.0.0-beta.8 (as of July 2025).
- [32] PowerDMARC. 2026. 4 Easy Ways To Rotate DKIM Keys. <https://powerdmarc.com/dkim-key-rotation-explained/>. Accessed: 27 February 2026.
- [33] Michael Raymond, Gillian Evers, Jan Ponti, Diya Krishnan, and Xiang Fu. 2023. Efficient zero knowledge for regular language. In *International Conference on Security and Privacy in Communication Systems*. Springer, 369–394.
- [34] N. Sakimura, J. Bradley, M. Jones, B. de Medeiros, and C. Mortimore. 2014. *OpenID Connect Core 1.0 incorporating errata set 1*. Technical Report. OpenID Foundation. Accessed: Oct 27, 2023.
- [35] Ken Thompson. 1968. Programming techniques: Regular expression search algorithm. *Commun. ACM* 11, 6 (1968), 419–422.
- [36] Alin Tomescu. 2025. Quadratic Arithmetic Programs (QAPs) and Rank-1 Constraint Systems (R1CS). <https://alinush.github.io/r1cs>. Accessed: 28 February 2026.
- [37] Matthias Wächs, Martin Schanzenbach, and Christian Grothoff. 2014. A Censorship-Resistant, Privacy-Enhancing and Fully Decentralized Name System. In *Proceedings of the 13th International Conference on Cryptology and Network Security (CANS 2014)*. <https://grothoff.org/christian/gns2014wachs.pdf>.
- [38] Collin Zhang, Zachary DeStefano, Arasu Arun, Joseph Bonneau, Paul Grubbs, and Michael Walfish. 2024. Zombie: Middleboxes that {Don't} snoop. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*. 1917–1936.

A Direct DFA-to-Circuit Translation

Direct DFA-to-Circuit Translation. The combined DFA was translated into a Circom circuit that explicitly encoded its state transition logic. For each input character, the circuit evaluated the current DFA state, checked the symbol against all valid transitions, and activated the corresponding next state. This approach required a large number of arithmetic components (e.g., IsEqual, AND) to represent the entire transition table, as illustrated in the simplified excerpt in Listing 2.

```

1 // ...
2 template BodyHashRegex(msg_bytes) {
3   signal input msg[msg_bytes];
4   signal output out;
5   // ...
6   signal states[num_bytes+1][35]; // Represents 35
    DFA states over num_bytes
7   // ...
8   for (var i = 0; i < num_bytes; i++) {
9     // ... (input character validation) ...
10    // Example of transition logic from state 0 for
        character 13 ('\r')
11    eq[0][i] = IsEqual();
12    eq[0][i].in[0] <== in[i];
13    eq[0][i].in[1] <== 13;
14    and[0][i] = AND();
15    and[0][i].a <== states[i][0]; // Is current state
        0 active?
16    and[0][i].b <== eq[0][i].out; // Is current char
        '\r'?
17    // states_tmp[i+1][1] would be set based on and[0]
        [i].out
18    // ... many similar lines for all transitions
        from all states ...

```

```

1509 19      states[i+1][1] <== MultiOR(2)([states_tmp[i+1][1]
1510      , from_zero_enabled[i] * and[0][i].out]); //
1511      Activate next state
1512 20  }
1513 21  // ...
1514 22  // Check if any accept state is reached
1515 23  component is_accepted = MultiOR(num_bytes+1);
1516 24  for (var i = 0; i <= num_bytes; i++) {
1517 25      is_accepted.in[i] <== states[i][34]; // State 34
1518      is an accept state
1519 26  }
1520 27  out <== is_accepted.out;
1521 28  // ...
1522 29  }

```

Listing 2: Encoding the entire transition table of a DFA in baseline.

B Detailed Discussion on Realistic System Model and Trust Assumptions

DNS Public Keys. Verifying DKIM signatures requires knowledge of the domain’s public signing key, which is published via the Domain Name System (DNS). These keys are subject to periodic rotation (typically every 6–24 months). In the current implementation, public keys are embedded in plaintext within the smart contract and verified by inspection. However, this model lacks cryptographic provenance.

To improve verifiability, we propose leveraging DNSSEC [6] to establish cryptographically authenticated DNS responses. Although DNSSEC adoption remains limited (notably, most major providers including Twitter do not support it), it provides a trust-minimized path to verifying key authenticity. Without DNSSEC, responses may be vulnerable to manipulation, allowing an adversary in a privileged network position to spoof the DKIM public key. To mitigate this, current contracts hard-code DNS keys, but this approach does not scale well across key rotations. Ultimately, on-chain verification of DNS records via DNSSEC or alternative trust frameworks is necessary for long-term resilience.

Censorship Resistance. To ensure censorship resistance, the system is designed such that no external party can unilaterally prevent proof generation. Notably, revocation of previously valid keys is only permitted if the corresponding private key is disclosed, which acts as a form of cryptographic revocation proof. While this behavior is uncommon in practice, some literature has proposed deliberate key leakage as a key rotation strategy to avoid provenance retention. In such cases, older keys remain valid indefinitely, unless explicitly revoked via private key disclosure. New keys are finalized only once all subsequent proofs use the updated key, preventing rollback attacks.

In scenarios with contested key updates, governance mechanisms may be invoked to resolve disputes, for example, through an on-chain n -of- m attestation quorum. Until DNSSEC adoption improves, the proposed approach anticipates the use of hybrid governance models to arbitrate key state transitions.

Decentralized Governance. We propose a composite governance framework for DNS key validation consisting of:

- Validator attestations from an EigenLayer-secured quorum,

- HTTPS-based TLS notary attestations from diverse endpoints,
- Enclave-based attestations from secure hardware environments,
- Threshold multisignature authorization.

While each mechanism has limitations when considered in isolation, the composite design ensures robust defense-in-depth. A successful adversary would be required to compromise all verification layers simultaneously. Furthermore, public verifiability of these attestations enables detection of compromise, facilitating rapid migration or system forking in adversarial conditions.

Comparative Advantage. In the absence of DNSSEC, proposed approach provides a stronger security model than traditional attestations by amortizing trust: a one-time attestation to a public DNS key enables the generation of verifiable ZKPs for an unbounded number of private email statements. This reduces the frequency and scope of consensus operations and allows for scalable and censorship-resistant provenance guarantees.

Self-Hosted DKIM Registries. For scenarios where reliance on the DNS hierarchy is undesirable, proposed scheme supports alternative key publication mechanisms. Specifically, domains may publish DKIM registries signed by an application-specific ECDSA keypair. This enables fully self-sovereign key management, suitable for protocol communities, organizations, or individual users.

Users may also rotate their email identity keys in a privacy-preserving manner. This is accomplished by sending a private email to a secondary address they control, attesting ownership of the original address. Upon DNS key rotation, a follow-up SNARK can prove prior access and authorize migration to the new key. Due to DKIM limitations, self-signed emails are not viable, so a secondary email address is required for this flow.

B.0.1 Mail Servers.

Sending Mail Server. By design, the DKIM private key is held by the domain’s outbound mail server, which enables it to sign any message purporting to originate from a user under that domain. This is consistent with the existing trust model of email infrastructure: users implicitly trust their email provider not to forge messages on their behalf. Thus, proposed scheme does not introduce new trust assumptions in this regard.

A more practical risk is that changes to the formatting of outgoing emails may invalidate previously functional regular expressions used for matching and proof construction. This necessitates periodic updates to the zk-circuit’s regex configuration and associated proving keys.

Receiving Mail Server. The receiving mail server has visibility into both the content and metadata of incoming emails. Consequently, any party with access to the receiving server may generate valid proofs for messages it observes. This limits the system’s ability to guarantee proof exclusivity for a recipient unless end-to-end encryption is employed. While the S/MIME standard could mitigate this by encrypting headers and payloads, adoption is currently limited.

This trust limitation is again congruent with existing email privacy expectations, where users trust providers (e.g., Gmail) not to misuse access to verification codes or sensitive messages.

B.0.2 Anonymity Considerations.

In applications using our constructions for anonymous credentials or pseudonymous identity proofs, nullifier-based constructions may enable linkability. For example, if a user posts an on-chain nullifier derived from the email body, the receiving mail server—having access to plaintext—may associate the nullifier with a specific user identity. Thus, anonymity and uniqueness guarantees are in tension: nullifiers help prevent Sybil attacks but may allow deanonymization by privileged servers.

To mitigate this, users are encouraged to employ encrypted email providers (e.g., Skiff, ProtonMail). However, providers such as ProtonMail modify MIME boundaries with nondeterministic identifiers, rendering message bodies unverifiable. In such cases, only header-based proofs are feasible.

For enhanced privacy, we support workflows where structured data (e.g., authorization codes) is embedded in encrypted email subject lines. If delivery is delayed sufficiently, this technique prevents correlation between content and recipient behavior.

C Full Proofs

In this section, we provide full proofs of Theorem 1, Theorem 3, and Theorem 4.

Unforgeability. We begin by proving Theorem 1.

PROOF. We prove the theorem via a sequence of game hops. Let $\text{Adv}_{i,\mathcal{A}} := \Pr[\text{Game}_{i,\mathcal{A}} = 1]$ denote the adversary's success probability in Game_i . Assuming there exists an adversary with non-negligible advantage in $\text{GameUnforge}_{\mathcal{A}}^{\text{Orig}}$, we show how to construct an algorithm that breaks the unforgeability of the underlying signature scheme SIG with non-negligible probability.

Game₀: We define Game_0 to be $\text{GameUnforge}_{\mathcal{A}}^{\text{Email}}$ from Figure 4. Hence, trivially $\text{Adv}_{0,\mathcal{A}} = \text{Adv}_{\text{GameUnforge}^{\text{Email}},\mathcal{A}}$.

Game₁: In this game, an adversary $\mathcal{B}$ is required to output the witness $w = (\sigma_{\text{DKIM}}, \ell_m, m)$ given the statement (pk_{srvr}, m^*) . Let X denote the knowledge extractor of the NIZK proof system for relation $\mathcal{R}_{\text{Email}}$. We construct $\mathcal{B}$ from $\mathcal{A}$ by extracting $w \leftarrow X(pk_{\text{srvr}}, m^*)$ because $\mathcal{A}$ can come up with satisfactory proof for m^* . Any failure of extraction would contradict the knowledge soundness of the NIZK. Therefore, $\text{Adv}_{0,\mathcal{A}} = \text{Adv}_{1,\mathcal{B}} + \text{negl}(\lambda)$.

Game₂: This proceeds as in Game_1 , except that, while verifying the forged witness $w' = (\sigma'_{\text{DKIM}}, \ell'_m, m')$, the challenger checks the m' against S_H and the adversary $\mathcal{B}$ loses if $m' \notin S_H$. Because of the collision-resistance of the underlying H , the chances of adversary $\mathcal{B}$ of finding an $m' \notin S_H$ but with valid proof, is negligible. Therefore, $\text{Adv}_{1,\mathcal{B}} = \text{Adv}_{2,\mathcal{B}} + \text{negl}(\lambda)$.

Now, combining the above games, we reach the fact that

$$\text{Adv}_{2,\mathcal{B}} = \text{Adv}_{\text{GameUnforge}^{\text{Email}},\mathcal{A}},$$

the adversary succeeds in Game_2 with a probability that is negligibly away from that in the $\text{GameUnforge}^{\text{Email}}$. Now observe that the Game_2 the adversary never wins unless they are able to come

up with a valid signature for a new message m' (not signed by the $\mathcal{O}^{\text{Mail}}$), such that $m' \notin S$ implying that they can break the unforgeability of the underlying signature scheme SIG. Hence the protocol is unforgeable. $\square$

Anonymity. Proving Theorem 3.

PROOF. We follow the same strategy of the game-based proof as for the unforgeability.

Game₀: We define Game_0 to be $\text{GameAnon}_{\mathcal{A}}^{\text{Email}}$ from Figure 5. Hence, trivially $\text{Adv}_{0,\mathcal{A}} = \text{Adv}_{\text{GameAnon}^{\text{Email}},\mathcal{A}}$.

Game₁: In this game, the proof π_{m_b} that is computed in line 7 of the Figure 5 is simulated. That is, instead of computing $\pi_{m_b} \leftarrow \mathcal{R}_{\text{Email}}.\text{Prv}(\sigma_{m_b}, \ell_{m_b}, m_b)$, it simulates the proof as $\pi_{m_b} \leftarrow \mathcal{R}_{\text{Email}}.\text{Sim}(\sigma_{m_b}, \ell_{m_b}, m_b)$. The computational indistinguishability of this game to Game_0 follows from the zero-knowledge property of the NIZK proof system, thus we have that $\text{Adv}_{0,\mathcal{A}} \leq \text{Adv}_{1,\mathcal{A}} + \text{negl}(\lambda)$.

Game₂: This game is the same as before with the only difference that the hash of the message m_b is sampled uniformly at random $h_{m_b} \xleftarrow{\$} \{0,1\}^k$, where k is the output length of the H . By the pseudorandomness of the H under our random oracle, this game hop is computationally indistinguishable, implying that $\text{Adv}_{1,\mathcal{A}} \leq \text{Adv}_{2,\mathcal{A}} + \text{negl}(\lambda)$.

As Game_0 is computationally indistinguishable from Game_2 , we have that

$$\text{Adv}_{2,\mathcal{A}} \leq \text{negl}(\lambda) \Rightarrow \text{Adv}_{\text{GameAnon}^{\text{Email}},\mathcal{A}} \leq \text{negl}(\lambda)$$

$\square$

Linkability. Proving Theorem 4.

PROOF. We follow the same strategy of the game-based proof as for the unforgeability and anonymity.

Game₀: We define Game_0 to be $\text{GameLink}_{\mathcal{A}}^{\text{Email}}$ from Figure 6. Hence, trivially $\text{Adv}_{0,\mathcal{A}} = \text{Adv}_{\text{GameLink}^{\text{Email}},\mathcal{A}}$.

Game₁: In this game, an adversary $\mathcal{B}$ is required to output the witnesses of the two proofs π'_0 and π'_1 . that is the $\mathcal{B}$ returns $w_b = (\sigma_{\text{DKIM}b}, \ell_{m_b}, m_b)$ for $b \in \{0,1\}$. We can construct $\mathcal{B}$ from $\mathcal{A}$ by extracting $w \leftarrow X(pk_{\text{srvr}}, m^*)$ because $\mathcal{A}$ can come up with satisfactory proof for m^* . Any failure of extraction would contradict the knowledge soundness of the NIZK. Therefore, $\text{Adv}_{0,\mathcal{A}} = \text{Adv}_{1,\mathcal{B}} + \text{negl}(\lambda)$.

Note that with Game_1 , we obtain an adversary $\mathcal{B}$ that can output two different $m'_0 \neq m'_1$ masked values from the message m . However, by construction, the masking procedure is consistently defined by the regex pattern via the function match. Hence, for a fixed message m and pattern rgx_{ptrn} , the resulting masked value m^* is uniquely determined. Thus, they must have been able to come up with two different m'_0 and m'_1 pre-images for the underlying hash function H in the DKIM, therefore, contradicting directly with the collision-resistance of the H , or breaking the unforgeability of the SIG by being able to verify a single signature for two different messages under the same key. Thus

$$\text{Adv}_{1,\mathcal{B}} \leq \text{negl}(\lambda) \Rightarrow \text{Adv}_{\text{GameLink}^{\text{Email}},\mathcal{A}} \leq \text{negl}(\lambda)$$

□

D Detailed Analysis

D.1 System Configuration

Table 3: Client-Side Proving Benchmark Environment

Property	Circom/Groth16	Noir/UltraHonk
CPU	Apple M4 Pro (14 cores)	
Memory	48 GB	
OS	Darwin 25.3.0, arm64	
Runtime	Node.js v24.0.1	
Compiler	circom 2.1.9	nargo 1.0.0-beta.5
Prover	snarkjs 0.7.5 (Groth16)	bb v0.84.0 (UltraHonk)
Trusted setup	ptau-22 [★]	N/A
Runs per config	3 (median reported)	5 (median reported)

[★] $2^{22} \approx 4.19 \times 10^6$ max constraints.

D.2 Circuit Complexity Scaling

Figure 17 shows that the number of R1CS constraints in Circom increases linearly with the total input size. The HEADER-HEAVY configuration (4096h/320b) lies on the same trend line, indicating that header and body bytes contribute approximately equally on a per-byte basis. The dashed line indicates the ptau-22 trusted setup limit (4.19×10^6 constraints). Figure 18 shows that the number of UltraPlonk gates in Noir also increases linearly with input size. In contrast to Circom, there is no fixed upper bound imposed by a trusted setup ceiling.

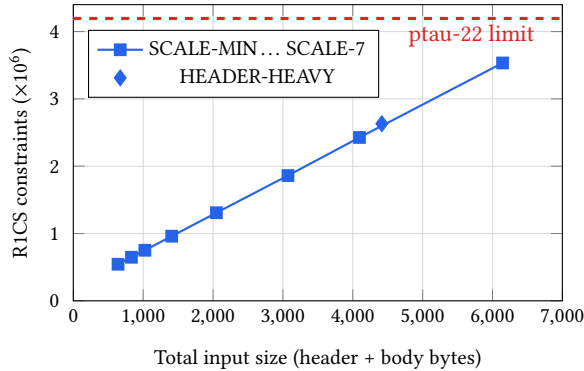


Figure 17: Circom: R1CS constraints vs. input sizes.

Table 4: Circuit size scaling data (both systems).

Config	Circom			Noir		
	H/B	Consts.	Comp.	H/B	Gates	Comp.
SCALE-MIN	384/256	543,299	8.4 s	—	—	—
SCALE-1	448/384	647,205	10.5 s	512/512	148,468	1.7 s
SCALE-2	512/512	750,853	12.4 s	512/768	170,437	2.0 s
SCALE-3	640/768	960,202	16.5 s	512/1024	192,406	2.2 s
SCALE-4	1024/1024	1,308,426	22.9 s	1024/1024	237,880	2.2 s
SCALE-5	1024/2048	1,860,876	34.1 s	1024/2048	325,756	2.9 s
SCALE-6	2048/2048	2,426,639	44.4 s	1024/4096	501,508	4.8 s
SCALE-7	2048/4096	3,533,585	68.3 s	2048/4096	592,456	5.4 s

Comp.=Compile time, Consts.=Number of constraints.

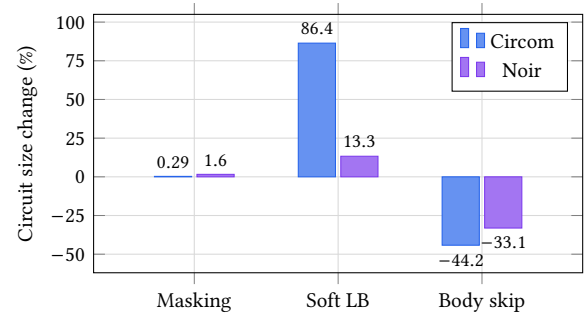


Figure 19: Cross-system feature comparison.

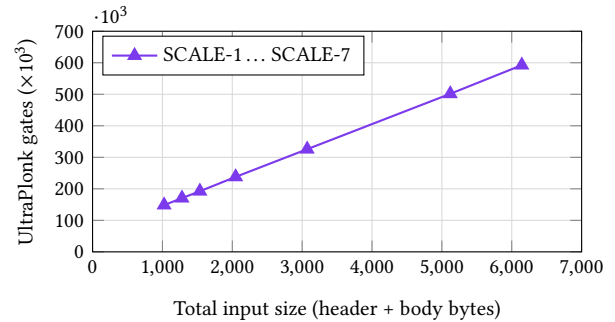


Figure 18: Noir: backend gates vs. input sizes

Figure 19 provides a direct side-by-side comparison. “Masking” corresponds to FEAT-FULL (Circom) versus FEAT-MASK (Noir); both introduce less than 2% overhead. “Soft LB” corresponds to FEAT-SOFT; the 86.4% overhead in Circom is confounded by the larger body allocation. “Body skip” compares FEAT-NOBODY (Circom, skipping body hashing) with FEAT-PARTIAL (Noir, partial hashing at 75%). The Circom FEAT-SOFT result is confounded by the increased body allocation (1408 vs. 768 bytes) and is not directly comparable to Noir’s +13.3% measurement.

Table 5 reports detailed measurements of proving time and witness generation time for different feature flags relative to the baseline [4]. Additional breakdowns are provided in Table 6. Masking constraints compose additively: header masking introduces exactly 1,280 constraints, body masking introduces exactly 1,536 constraints, and enabling both equals their sum.

Table 5: Feature flag impact on proving time (median).

Feature	Circom (Groth16)			Noir (UltraHonk)		
	Prove	WitGen	Δ	Prove	WitGen	Δ
BASE	8.30 s	1.80 s	—	5.15 s	396 ms	—
NOBODY	6.98 s	1.31 s	−15.9%	—	—	—
HMASK	8.49 s	1.80 s	+2.2%	—	—	—
BMASK	8.25 s	1.81 s	−0.7%	—	—	—
FULL	8.28 s	1.80 s	−0.3%	—	—	—
SOFT [†]	16.37 s	2.66 s	+97.1%	5.55 s	474 ms	+7.8%
PARTIAL	—	—	—	3.38 s	271 ms	−34.4%
MASK	—	—	—	5.33 s	636 ms	+3.4%
ADDR	—	—	—	3.12 s	331 ms	−39.4%

[†] Circom FEAT-SOFT uses maxBodyLength=1408 (others use 768).

When ignoreBodyHashCheck= 1, body masking becomes a compile-time no-op, as the body is not loaded into the circuit. This confirms the correctness of the circuit’s conditional compilation logic.

Table 6: Circom masking constraints are perfectly additive.

Config	Constraints	Delta from BASE
FEAT-BASE	960,202	—
FEAT-HMASK	961,482	+1,280
FEAT-BMASK	961,738	+1,536
FEAT-FULL (H+B)	963,018	+2,816 = 1280 + 1536
FEAT-NOBODY	535,950	−424,252
FEAT-NOBODY-BMASK	535,950	= NOBODY [†]
FEAT-NOBODY-HMASK	537,230	= NOBODY + 1,280
FEAT-NOBODY-FULL	537,230	= NOBODY-HMASK [★]

[†] body mask is compile-time no-op. [★] body mask no-op.

Our Noir implementation supports both RSA-1024 and RSA-2048 key sizes. Circom’s standard RSA library enforces $n \times k > 2048$, and therefore RSA-1024 is supported only in Noir. We evaluate the impact of the RSA public key size on both prover and verifier complexity, as reported in Table 7. RSA-2048 costs approximately 10% more in gates and proving time vs. RSA-1024.

Table 7: RSA key size impact (Noir, verify_email, 512h/1024b).

Config	Gates	Prove	WitGen	Verify
RSA-1024	173,847	4.78 s	303 ms	1.93 s
RSA-2048	192,406	5.32 s	380 ms	2.07 s
Δ	−9.6%	−10.1%	−20.3%	−6.6%

Figure 20 shows RSA-2048 approximately consumes 10% more gates and proving time compared to RSA-1024. The results related to Noir implementation; Circom’s email-verifier template enforces $n \times k > 2048$.

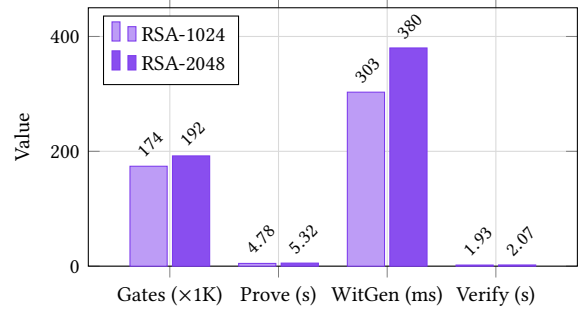
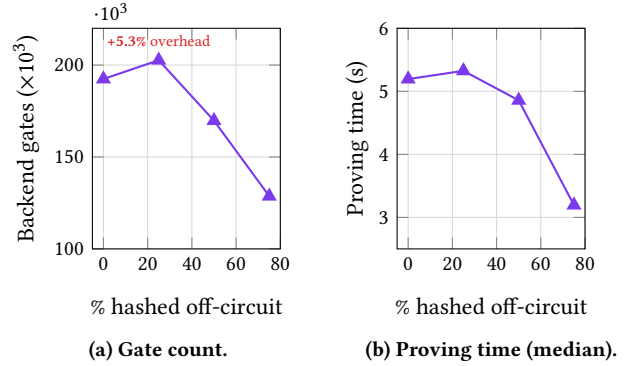
**Figure 20: RSA-2048 vs. RSA-1024 costs.**

Figure 21 shows the Noir SHA-256 precomputation sweep. The partial_hash circuit template introduces a fixed overhead due to additional partial-hash verification logic. At 25% precomputation, this overhead exceeds the gate savings obtained by reducing maxBodyLength from 1024 to 768 bytes, resulting in a net increase in gate count. The crossover point occurs between 25% and 50% precomputation. At 75% precomputation, the circuit achieves a reduction of −33.1% in gate count and −38.5% in proving time.

**Figure 21: Impact of SHA-256 pre-computation in Noir on gate count and proving time.**

In Circom, the precomputation position (10%, 50%, 80%) affects only the *input data*, i.e., the location at which the SHA intermediate state is injected, and does not modify the *circuit topology*. All PRECOMP configurations with maxBodyLength=768 yield identical constraint counts. Reducing the number of constraints requires explicitly lowering maxBodyLength. This behavior is expected, as the circuit is parameterized by maximum input sizes rather than by the actual precomputation offset.

Table 8: Circom SHA precomputation: constraints and proving times.

Config	Pos.	Body	Consts.	Δ	Prove	WitGen
PRECOMP-NONE	0%	768	960,202	—	8.19 s	1.80 s
PRECOMP-EARLY	10%	768	960,202	0%	8.16 s	1.79 s
PRECOMP-MID	50%	768	960,202	0%	8.26 s	1.80 s
PRECOMP-LATE	80%	768	960,202	0%	8.21 s	1.81 s
PRECOMP-MID-SM	50%	448	787,752	-18.0%	7.64 s	1.59 s
PRECOMP-LATE-SM	80%	320	719,080	-25.1%	not proven*	

* PRECOMP-LATE-SMALL compiled successfully but proving was not completed.
Consts.=Number of Constraints. Pos.=Position.

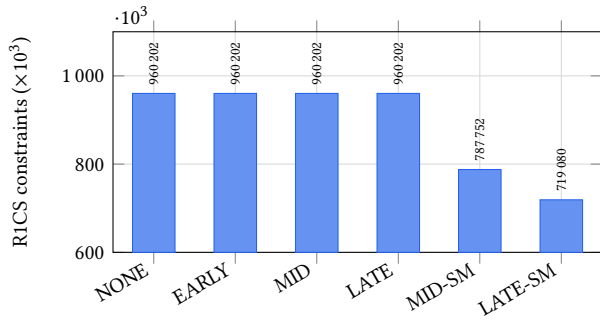


Figure 22: Circom precomputation effect: only reduced body lowers constraints.

Table 9: Circom/Groth16 verification time across all configs (median, ms).

Config	Verify (ms)	Config	Verify (ms)
SCALE-MIN	182	FEAT-BASE	185
SCALE-1	184	FEAT-NOBODY	181
SCALE-2	188	FEAT-HMASK	186
SCALE-3	186	FEAT-BMASK	185
SCALE-4	184	FEAT-SOFT	184
SCALE-5	181	FEAT-FULL	186
SCALE-6	206	PRECOMP-NONE	184
SCALE-7	205	PRECOMP-MID-SM	184
HDR-HEAVY	206		
Range: 181 ms–206 ms, Mean: 188 ms			

D.3 Prover Complexity

Figure 23 shows that proving time scales approximately linearly with circuit size in both systems. Circom/Groth16 exhibits a steeper slope and a visible step between 960 k and 1.3 M constraints, corresponding to a ptau power-of-two boundary.

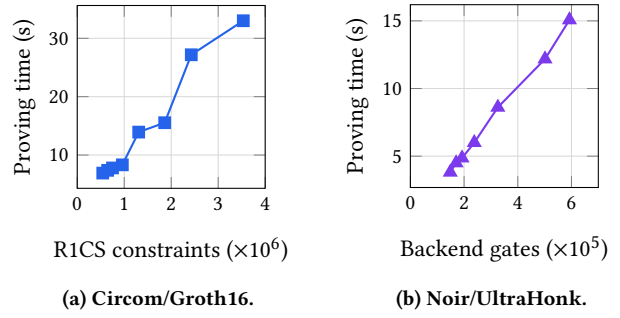


Figure 23: Proving time vs circuit size.

For a direct comparison, Figure 24 reports proving time at identical input sizes. Overall, Noir/UltraHonk achieves **2.0–2.3× faster** proving than Circom/Groth16. The compared configurations are Circom SCALE-2/4/5/7 and Noir SCALE-1/4/5/7, evaluated at identical header and body sizes.

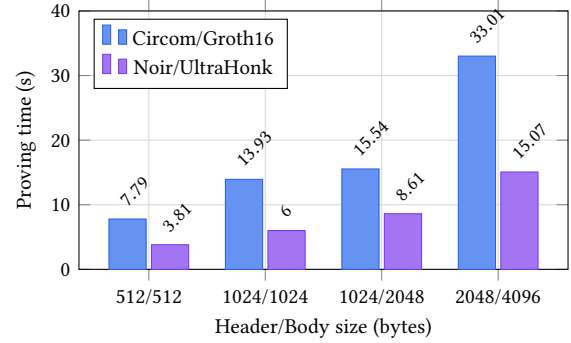


Figure 24: Head-to-head proving times at matching input sizes.

Figure 25 provides a detailed comparison of witness generation time across the same configurations. It shows Noir generates witnesses **3.2–6.3× faster** than Circom across all input sizes. Circom uses WASM-based witness computation; Noir uses native code via nargo.

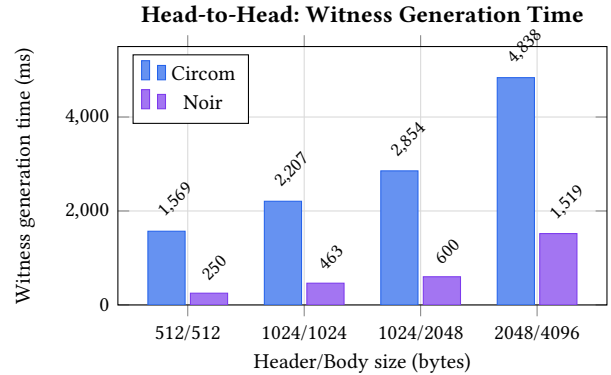


Figure 25: Witness generation time: Noir vs. Circom.

D.4 Verifier and Communication Complexity

Table 10: Proof size and verification cost comparison.

Property	Groth16	UltraHonk	Ratio
Proof size	~805 B	14,080 B	Groth16 17.5× smaller
Verification (small)	185 ms	1.69 s	Groth16 9.1× faster
Verification (large)	205 ms	5.80 s	Groth16 28.3× faster
Trusted setup	Required (ptau ceremony)	Not required	UltraHonk advantage
Max circuit	$\sim 4.19 \times 10^6$ (ptau-22)	No hard limit	UltraHonk advantage

D.5 Setup and Compilation Performance

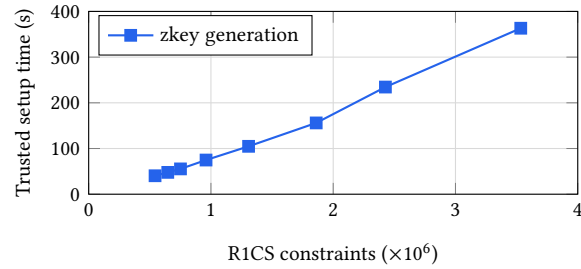


Figure 26: Groth16 trusted setup (zkey generation) scales linearly with constraint count. The largest circuit (3,533,585 constraints) requires over 6 minutes. UltraHonk has no equivalent cost.

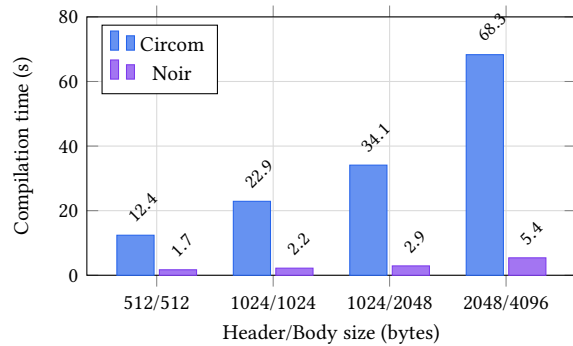


Figure 27: Noir compiles 7.3–12.7× faster than Circom at every input size. Circom compilation involves R1CS constraint generation and WASM output; Noir uses native ACIR compilation via nargo.

Table 11: Head-to-head comparison at matching input sizes (median values).

Size	System	Circuit Size	Prove (s)	WitGen (ms)	Verify (ms)
512/512	Circom	750,853 R1CS	7.79	1569	188
	Noir	148,468 gates	3.81	250	1694
1024/1024	Circom	1,308,426 R1CS	13.93	2207	184
	Noir	237,880 gates	6.00	463	2353
1024/2048	Circom	1,860,876 R1CS	15.54	2854	181
	Noir	325,756 gates	8.61	600	3326
2048/4096	Circom	3,533,585 R1CS	33.01	4838	205
	Noir	592,456 gates	15.07	1519	5799

E Data Quality and Variance

E.1 Circom Proving Time Variance

Circom benchmarks use 3 runs per configuration. Table 12 shows the spread across runs.

Table 12: Circom proving time variance (3 runs, milliseconds).

Config	Min	Median	Max	CV (%)
SCALE-MIN	6908	6911	6920	0.1
SCALE-1	7371	7383	7409	0.3
SCALE-2	7647	7790	7836	1.3
SCALE-3	8289	8320	8400	0.7
SCALE-4	13889	13930	14004	0.4
SCALE-5	15447	15544	15559	0.4
SCALE-6	27091	27184	27895	1.6
SCALE-7	32219	33015	33714	2.3
FEAT-BASE	8245	8303	8339	0.6
FEAT-NOBODY	6963	6977	7072	0.8
FEAT-SOFT	16262	16366	16433	0.5
PRECOMP-NONE	8188	8189	8236	0.3
PRECOMP-MID-SM	7643	7644	7648	0.0

CV = coefficient of variation (stddev / mean $\times$ 100%). All configs show CV < 3%, indicating stable measurements.

E.2 Noir Proving Time Variance

Noir benchmarks use 5 runs per configuration.

Table 13: Noir proving time variance (5 runs, milliseconds).

Config	Min	Median	Max	CV (%)
SCALE-1	3693	3813	4439	8.5
SCALE-2	4327	4501	4587	2.7
SCALE-3	4587	4864	5583	7.7
SCALE-4	5965	6000	6328	2.4
SCALE-5	8360	8610	9195	3.5
SCALE-6	11270	12179	13559	7.3
SCALE-7	14552	15070	15483	2.9
RSA-1024	4392	4780	5117	6.6
RSA-2048	5229	5319	5604	3.0
FEAT-BASE	4762	5153	5569	5.9
FEAT-PARTIAL	3202	3378	4224	10.7
FEAT-SOFT	5471	5550	6786	8.5
PRECOMP-0	4784	5194	5405	4.7
PRECOMP-75	2943	3194	3247	3.5

Noir shows higher variance (CV up to 10.7%) than Circom (< 3%), likely due to UltraHonk’s memory allocation patterns and OS-level scheduling on Apple Silicon.

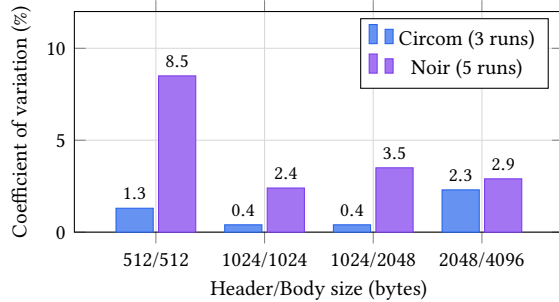


Figure 28: Proving time coefficient of variation at matching input sizes. Circom/Groth16 is more deterministic (< 3% CV), while Noir/UltraHonk shows higher variance (up to 8.5%).

E.3 Circom Public Output Sizes

Table 14: Circom public output sizes by feature configuration.

Config	Public Size (B)	Notes
BASE / NOBODY / SOFT	~874	Standard public signals
FEAT-HMASK	5244	Includes masked header bytes
FEAT-BMASK	6370	Includes masked body bytes
FEAT-FULL	10740	Both masked header + body

Masking features expose the masked content as public outputs, increasing the public signal size by 5–12×.

F ZK Regex Benchmarks

This section presents standalone benchmarks for the zk-regex compiler described in §3.2, evaluating the scaling claims made in §5. We compare three backends – DFA encoding (Circom/Groth16),

NFA encoding (Circom/Groth16), and NFA encoding (Noir/UltraHonk) – across 15 regex patterns of increasing complexity, at input capacities from 64 to 512 bytes. All data and reproduction scripts are available in benchmarks/ of the zk-regex repository.

F.1 Test Suite and Methodology

We select 15 patterns spanning four complexity tiers (Table 15): four *simple* (literals, basic character classes), four *medium* (alternation, non-capturing groups, real-world identifiers), four *complex* (DKIM headers, URL matching, capture groups), and three *NFA-only* (lazy quantifiers, unbounded ranges, nested quantified groups).

Table 15: Regex test suite. “DFA” indicates the DFA compiler can handle the pattern. Complexity is assigned by feature set, not pattern length.

Pattern	Regex	Complexity	Category	DFA
simple	a*b	simple	synthetic	✓
literal	hello	simple	synthetic	✓
char_class	[a-z]+	simple	synthetic	✓
alternation	foo bar baz	simple	synthetic	✓
nested_nc	(?:a(?:b c)d)+	medium	synthetic	✓
alphanumeric	[A-Za-z0-9_]+	medium	real-world	✓
email_basic	[a-zA-Z0-9._%+-]+@...	medium	real-world	✓
subject	(?:\r\n *)subject:...\r\n	medium	real-world	✓
url	(?:https? ftp)://...	complex	real-world	✓
body_hash	...bh=[a-zA-Z0-9+/=]+;	complex	real-world	✓
email_addr	(?:\r\n *)to:...@...\r\n	complex	real-world	✓
fixed_range	[a-z]{3}[0-9]{2,4}[A-Z]{1,2}	complex	synthetic	—
lazy_quant	<.*?>	NFA-only	synthetic	*
unbounded	[a-z]{3,}	NFA-only	synthetic	*
nested_qg	(?:[a-z]+[0-9]+)+	NFA-only	synthetic	*

* DFA compiler accepts these patterns unexpectedly; see §F.8.

Input scaling. Each input length is filled with *regex-matching content*, not zero-padded short strings. Three strategies are used: *repeat* (tile a template string), *extend* (grow a variable-length match portion), and *pad-with-match* (place an anchored template at the start, fill the remainder with safe filler). This ensures benchmarks measure the cost of real regex work at each capacity.

Methodology. Each pattern is benchmarked at four input capacities (64, 128, 256, 512 bytes) across all three backends. We report the median of 5–10 runs per backend per pattern, with standard deviations. Hardware: Apple M4 Max, 14 cores, 36 GB RAM, macOS Darwin 25.3.0. Software: Bun 1.3.10, circom 2.2.2, nargo 1.0.0-beta.5, Barretenberg v0.84.0.

F.2 NFA Transition Structure

The NFA compiler generates an epsilon-free NFA and encodes it differently for each backend. In **Circom**, each byte position iterates all NFA transitions; the generated template declares the count as `var numTransitions = N`. In **Noir**, the NFA is expanded into a byte-level sparse array of size N where each possible (state, byte, next_state) triple becomes a separate entry. These counts differ: a single Circom NFA transition on `[a-z]` expands to 26 Noir sparse entries (one per letter). Table 16 shows both counts for all 15 patterns.

Table 16: NFA transition counts extracted from generated circuit files. Circom transitions (numTransitions) count NFA-level tuples; Noir sparse entries count the expanded byte-level lookup table. Circom NFA R1CS at 64-byte input is shown for correlation.

Complexity	Pattern	Circom Trans.	Noir Entries	NFA R1CS	Noir Gates
simple	char_class	3	78	51,892	49,322
	simple	6	6	41,096	48,964
	literal	6	6	41,092	48,962
	alternation	9	9	51,892	48,977
medium	nested_nc	5	6	44,692	48,962
	alphanumeric	12	189	127,492	49,877
	email_basic	37	487	339,892	51,557
	subject	43	688	441,901	63,031
complex	url	31	211	217,496	49,989
	body_hash	66	910	630,301	65,819
	email_addr	74	1,062	779,101	65,275
	fixed_range	12	248	151,192	50,472
NFA-only	lazy_quant	20	501	185,092	51,641
	unbounded	5	130	73,492	49,582
	nested_qg	6	124	84,292	49,552

The Noir sparse entry count is the better proxy for Circom’s actual per-byte circuit cost, because it counts the expanded byte-level transition checks that Circom must iterate. The correlation between Circom NFA R1CS and Noir sparse entries is clear: patterns with 6 entries have ~41,000 R1CS, patterns with ~500 entries have ~340,000–440,000 R1CS, and patterns with 1,000+ entries reach ~780,000 R1CS. Meanwhile, **Noir gates remain nearly flat** (48,962–65,819) across the same 6–1,062 entry range – confirming that the sparse-array lookup cost is independent of table size.

F.3 Circuit Complexity

Figure 29 shows circuit size at 64-byte input across all three backends. Two fundamental scaling behaviors emerge.

Circom NFA scales as $O(\text{match_length} \times \text{total_transitions})$. As stated in §5, the Circom backend iterates all NFA transitions at each byte position. The benchmark data confirms this: Circom NFA R1CS constraints range 19× across patterns (41,092–779,101) at fixed 64-byte input, directly correlated with the Noir sparse entry count (Table 16). The three most complex patterns (body_hash: 910 entries, email_addr: 1,062) require >15× more constraints than simple (6 entries).

Table 17 provides the complete cross-backend comparison.

Table 17: DFA vs Circom NFA constraint counts and Noir gates at 64-byte input. The NFA/DFA column shows the NFA blowup factor. fixed_range fails DFA compilation (accept-node limitations).

Complexity	Pattern	DFA R1CS	NFA R1CS	NFA/DFA	Noir Gates
simple	simple	2,131	41,096	19.3	48,964
	literal	3,561	41,092	11.5	48,962
	char_class	3,821	51,892	13.6	49,322
	alternation	4,861	51,892	10.7	48,977
medium	nested_nc	3,496	44,692	12.8	48,962
	alphanumeric	8,696	127,492	14.7	49,877
	email_basic	15,651	339,892	21.7	51,557
	subject	39,822	441,901	11.1	63,031
complex	url	15,326	217,496	14.2	49,989
	body_hash	60,295	630,301	10.5	65,819
	email_addr	24,589	779,101	31.7	65,275
	fixed_range	—	151,192	—	50,472
NFA-only	lazy_quant	32,486	185,092	5.7	51,641
	unbounded	4,471	73,492	16.4	49,582
	nested_qg	7,396	84,292	11.4	49,552

Noir scales as $O(\text{match_length} \times \text{lookup_cost})$ with effectively constant lookup. Also as stated in §5, Noir replaces per-transition iteration with a single sparse-array lookup per byte, keyed by $\text{key} = s + b \cdot 257 + s' \cdot 257^2$. The total gate count ranges only 1.34× across all 15 patterns (48,962–65,819). Twelve of fifteen patterns cluster in the narrow band 48,962–51,641 gates; the three with slightly higher counts (subject, body_hash, email_addr) share large character-class alphabets and capture-group structures that expand the fixed lookup-table overhead.

The most direct test of “constant lookup cost” is how circuit size scales with input capacity (§F.4).

F.4 Scaling with Input Length

We measure circuit size at four input capacities: 64, 128, 256, and 512 bytes. Figure 30 shows three representative patterns.

DFA scales linearly with input length. DFA R1CS doubles at each doubling of input (ratio 1.99×), as expected from in-circuit DFA simulation where each byte adds a fixed per-state cost.

Circom NFA is constant within a template capacity. Circom NFA produces identical circuit sizes from 64 to 256 bytes, then steps up at 512 bytes when the compiler generates a new template with larger max_byte_len. This step-function behavior confirms the architecture described in §3.2: the circuit is compiled once for a fixed capacity, and all inputs within that capacity share the same constraint structure.

Noir is constant across all input capacities. Noir produces identical gate counts from 64 to 512 bytes for every pattern tested. Unlike Circom NFA, there is no step-up at 512 bytes – the circuit structure is entirely independent of input capacity. This is the strongest possible evidence for the constant-cost sparse-array lookup: the marginal per-byte cost is zero.

Marginal per-byte cost. Table 18 shows the additional circuit cost per byte when the template capacity increases from 256 to 512 bytes. This isolates the per-byte verification cost from the fixed overhead (lookup table setup, input validation, state initialization).

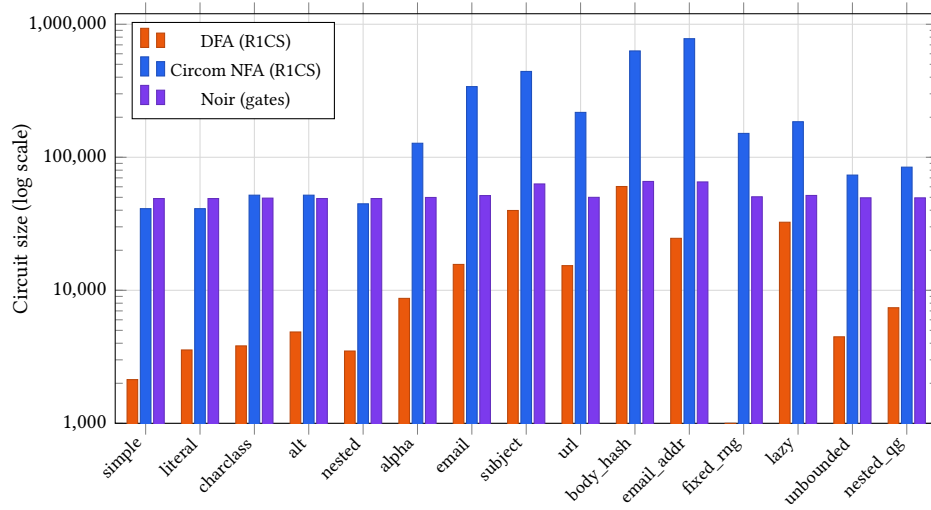


Figure 29: Circuit size at 64-byte input across 15 patterns (log scale). DFA R1CS constraints are omitted for fixed_range (compilation failure). Circom NFA R1CS spans a 19× range across patterns; Noir gates span only 1.34×.

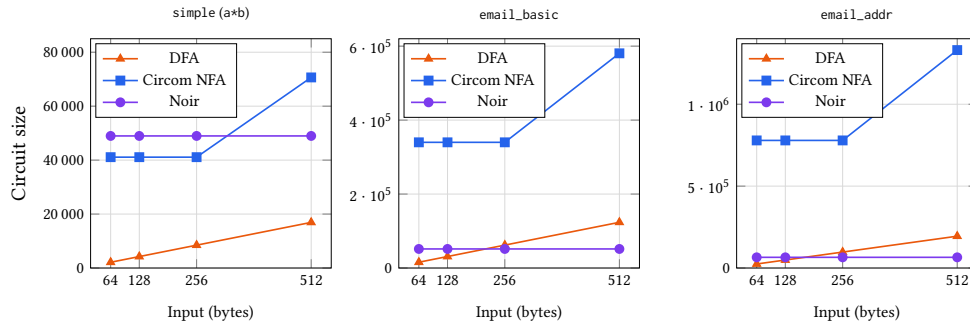


Figure 30: Circuit size scaling with input length for three representative patterns. DFA (orange) grows linearly with a 1.99× ratio at each doubling. Circom NFA (blue) is constant from 64 to 256 bytes (same compiled template), with a step increase at 512 bytes when a new template is generated. Noir (purple) is constant across all four input capacities.

Table 18: Marginal per-byte cost when template capacity increases from 256 to 512 bytes. Noir gate counts are identical at both capacities (zero marginal cost). The Circom range (18.6×) reflects iteration over all NFA transitions.

Complexity	Pattern	Circom NFA (R1CS)			Noir (gates)		
		@256 B	@512 B	R1CS/byte	@256 B	@512 B	gates/byte
simple	simple	41,096	70,654	115.5	48,964	48,964	0.0
	char_class	51,892	89,082	145.3	49,322	49,322	0.0
medium	email_basic	339,892	580,602	940.3	51,557	51,557	0.0
	subject	441,901	755,203	1,223.8	63,031	63,031	0.0
complex	body_hash	630,301	1,076,739	1,744.7	65,819	65,819	0.0
	email_addr	779,101	1,330,691	2,154.6	65,275	65,275	0.0

In Circom, the marginal per-byte cost ranges from 115.5 R1CS/byte (simple, 6 sparse entries) to 2,154.6 R1CS/byte (email_addr, 1,062 entries) – an 18.6× range that mirrors the NFA transition count. In Noir, the marginal cost is zero – gate counts are identical at 256 and 512 bytes for every pattern. This is the most direct evidence that Noir’s circuit structure is entirely independent of input capacity:

the sparse-array lookup table is compiled once and does not grow with the input buffer.

F.5 Proving Performance

Figure 12 compares proving times at 64-byte input. We use Groth16 for Circom (DFA and NFA) and UltraHonk for Noir.

Noir is 5–15× faster than Circom NFA (Groth16). The speedup ranges from 4.8× (char_class: 298 ms vs. 1,442 ms) to 15.5× (email_addr: 548 ms vs. 8,469 ms), and correlates with regex complexity. For simple patterns, the advantage comes mainly from UltraHonk’s faster per-gate proving. For complex patterns, it compounds: both the circuit is smaller *and* the prover is faster per gate.

Noir is 2–6× faster than DFA. Even compared to the DFA’s much smaller R1CS circuits, Noir proves faster thanks to UltraHonk’s efficient backend. The speedup at 512 bytes grows further: 3.9× for simple and 7.9× for body_hash.

Context: regex-only vs end-to-end speedup. The 5–15× Noir speedup measured here applies to the *regex component only*. In our full pipeline (§6), SHA-256 hashing dominates circuit size and both backends have similar hashing costs, yielding the 1.8–2.3× end-to-end speedup reported in §6. The regex component exhibits a larger speedup because the bottleneck – per-transition iteration – is exactly what Noir’s sparse-array encoding eliminates.

Table 19: Proving time scaling by input length (ms). Circom NFA is stable from 64–256 bytes (same template), stepping up at 512 bytes. Noir remains stable across all input sizes.

Pattern	Input (B)	DFA	Circom NFA	Noir	Noir/NFA
simple	64	933	1,450	281	5.2
	128	992	1,502	282	5.3
	256	1,034	1,534	282	5.4
	512	1,108	1,813	283	6.4
email_basic	64	2,064	4,197	373	11.3
	128	1,193	4,238	371	11.4
	256	1,392	4,230	373	11.3
	512	3,351	7,150	369	19.4
email_addr	64	1,322	8,469	548	15.5
	128	1,437	7,746	544	14.2
	256	1,847	8,380	547	15.3
	512	2,670	13,872	544	25.5

F.6 Verification and Proof Size

Table 20 shows the Noir backend details at 64-byte input. UltraHonk produces constant-size proofs of 14,080 B (≈ 13.8 kB) with verification time of ≈ 32 ms across all patterns.

Table 20: Noir/UltraHonk verification details at 64-byte input. Proof size is constant at 14,080 B for all patterns. Verification time is constant at ≈ 32 ms.

Pattern	ACIR	Gates	Prove (ms)	Verify (ms)
simple	26,417	48,964	281 $\pm$ 1	31 $\pm$ 1
literal	26,415	48,962	285 $\pm$ 3	33 $\pm$ 1
char_class	26,487	49,322	298 $\pm$ 2	32
alternation	26,418	48,977	284 $\pm$ 2	32 $\pm$ 1
nested_nc	26,415	48,962	284 $\pm$ 2	33 $\pm$ 1
alphanumeric	26,598	49,877	316 $\pm$ 1	32 $\pm$ 1
email_basic	26,896	51,557	373 $\pm$ 2	33 $\pm$ 1
subject	38,570	63,031	458 $\pm$ 3	31 $\pm$ 1
url	26,622	49,989	322 $\pm$ 2	31 $\pm$ 1
body_hash	40,008	65,819	586 $\pm$ 5	31 $\pm$ 1
email_addr	38,944	65,275	548 $\pm$ 5	32 $\pm$ 1
fixed_range	26,957	50,472	331 $\pm$ 2	32
lazy_quant	26,910	51,641	373 $\pm$ 2	32 $\pm$ 1
unbounded	26,539	49,582	305 $\pm$ 2	31 $\pm$ 1
nested_qg	26,533	49,552	302 $\pm$ 2	31 $\pm$ 1

The ACIR opcode counts show a bimodal distribution: 12 patterns cluster around 26,400–27,000 opcodes, while three (subject, body_hash, email_addr) require 38,500–40,000 opcodes. These three share large character-class alphabets and capture-group structures. Despite this, the resulting gate overhead is modest (1.34× worst case).

Note on verification time. The ≈ 32 ms verification measured here for 49,000–66,000 gate circuits is substantially lower than the 1.7 s–5.8 s range reported in §6 for full circuits with 148,000–592,000 gates. This difference likely reflects invocation overhead (CLI process startup, file I/O, verification-key loading) that dominates at small circuit sizes, as well as potential differences in recursive vs non-recursive verification modes.

F.7 Memory Usage

Table 21 compares compilation memory across backends for selected patterns at 64-byte input.

Table 21: Compilation memory (MB) at 64-byte input. Circom NFA requires up to 25× more memory than DFA for complex patterns. Noir compilation memory is more uniform.

Pattern	DFA Compile	NFA Compile	Noir Compile
simple	28	194 $\pm$ 9	385 $\pm$ 12
literal	36 $\pm$ 1	189 $\pm$ 8	384 $\pm$ 13
char_class	30 $\pm$ 1	157 $\pm$ 1	390 $\pm$ 15
email_basic	91 $\pm$ 2	1,064 $\pm$ 37	394 $\pm$ 26
subject	206 $\pm$ 7	1,923 $\pm$ 93	452 $\pm$ 50
body_hash	200 $\pm$ 2	2,737 $\pm$ 93	460 $\pm$ 26
email_addr	119 $\pm$ 4	2,968 $\pm$ 228	485 $\pm$ 38

Circom NFA compilation memory varies widely (157 MB to 2,968 MB), scaling with NFA transition count. Noir compilation memory is more uniform (372 MB to 485 MB), reflecting the sparse-array compression. However, Noir’s *proving* memory is significant: 597 MB to 1,335 MB, dominated by UltraHonk’s polynomial commitment operations.

F.8 DFA Compatibility

We tested all 15 patterns against the DFA compiler. Only `fixed_range_quantifier` (`[a-z]{3}[0-9]{2,4}[A-Z]{1,2}`) fails, producing a “DFA accept nodes” error from multiple bounded range quantifiers. Surprisingly, the three patterns classified as *NFA-only* – lazy quantifier (`<.*?>`), unbounded range (`[a-z]{3,}`), and nested quantified groups (`((?:[a-z]{10-9})+)`) – all compile under the DFA compiler.

This has two implications. First, the DFA compiler handles a broader regex subset than expected, likely by internally expanding quantifiers and converting lazy to greedy semantics. Alternations (`|`) are handled natively by DFAs and are not a DFA limitation. Second, the true differentiator for the NFA approach is not *whether* a pattern compiles, but that the NFA-based approach preserves exact match semantics (e.g., lazy vs. greedy) that the DFA expansion may silently alter. The genuine DFA limitation is *bounded range*

quantifiers ($\{n, m\}$) with multiple ranges, which cause DFA state explosion.

F.9 Summary

Table 22: ZK regex backend comparison summary at 64-byte input. Ranges span all 15 tested patterns.

Metric	DFA (Groth16)	Circom NFA (Groth16)	Noir (UltraHonk)
Constraints/Gates	2,131–60,295	41,092–779,101	48,962–65,819
Variation	28×	19×	1.34×
Marginal cost/byte	—	115–2,155 R1CS/B	0 gates/B
Per-byte variation	—	18.6×	N/A (zero)
Prove time	933 ms to 2,574 ms	1,410 ms to 8,469 ms	281 ms to 586 ms
Verify time	<1 ms (Groth16)	<1 ms (Groth16)	≈32 ms
Proof size	128 B (Groth16)	128 B (Groth16)	14,080 B
Regex coverage	Most patterns	All patterns	All patterns
Match semantics	Greedy only (DFA)	Exact (NFA)	Exact (NFA)

These results validate the scaling analysis in §5:

- **Circom scales as $O(\text{match_length} \times \text{total_transitions})$.** Circom NFA R1CS constraints vary 19× across patterns at fixed input and the marginal per-byte cost varies 18.6×, proportional to the NFA transition count. Within a compiled template, circuit size is constant; it steps up when a new template is generated for larger capacity.
- **Noir’s circuit is entirely independent of input capacity.** Gate counts vary only 1.34× across patterns, and the marginal per-byte cost is *zero* – Noir circuits are identical from 64 to 512 bytes. This confirms that the RLC-keyed sparse-array lookup produces a fixed circuit structure regardless of both NFA size and input capacity.
- **Noir’s advantage grows with pattern complexity.** The Noir-over-Circom NFA (Groth16) proving speedup scales from 4.8× (simplest pattern) to 15.5× (most complex), and from 15× at 64 bytes to 26× at 512 bytes for complex patterns, because Noir proving time stays flat while Circom NFA increases at higher input capacities.
- **Circuit size independence from regex complexity holds for Noir, not Circom.** The claim that “circuit complexity is independent of regex size” (§3.2) is confirmed for the Noir backend (1.34× range, zero marginal cost) but does not hold for the Circom NFA backend (19× range, 18.6× marginal variation). This distinction should be qualified when stated in general terms.

The cost of Noir’s advantages is a larger proof (≈14 kB vs. 128 B for Groth16) and higher verification time (≈32 ms vs. sub-millisecond), which may be acceptable for applications where prover efficiency is the primary bottleneck.

F.10 Reproducibility

Table 23: ZK Regex benchmark machine specifications.

Component	Specification
CPU	Apple M4 Max
Cores	14
Memory	36 GB unified
OS	macOS (Darwin 25.3.0)
Architecture	arm64 (Apple Silicon)

Table 24: Software versions used in ZK Regex benchmarks.

Tool	Circom Stack	Noir Stack
Compiler	circom 2.2.2	nargo 1.0.0-beta.5
Prover	snarkjs (Groth16)	bb (barretenberg) v0.84.0
Backend	Groth16 (BN254)	UltraHonk
Runtime	Bun 1.3.10	Bun 1.3.10
Trusted Setup	Hermez ptau	N/A

Software versions.

Reproduction commands.

Full benchmark suite (all providers, all patterns)
cd benchmarks && bun run bench

Individual providers
bun run bench:circom-v1 # DFA encoding via git worktree
bun run bench:circom-v2 # NFA encoding (current compiler)
bun run bench:noir # NFA encoding + UltraHonk

Generate LaTeX/Markdown outputs
bun run generate-outputs

Configuration is in benchmarks/config/benchmark.json (3 warmup runs, 5–10 timed runs per backend per pattern) and benchmarks/config/patterns (15 pattern definitions with input-scaling strategies).

G Reproducibility

G.1 Machine Specifications

Table 25: Benchmark machine specifications.

Component	Specification
CPU	Apple M4 Pro
Cores	14 (10 performance + 4 efficiency)
Memory	48 GB unified
OS	macOS (Darwin 25.3.0)
Architecture	arm64 (Apple Silicon)

G.2 Software Versions

Table 26: Software versions used in benchmarks.

Tool	Circom Stack	Noir Stack
Compiler	circom 2.1.9	nargo 1.0.0-beta.5
Prover	snarkjs 0.7.5	bb (barretenberg) v0.84.0
Backend	Groth16 (BN254)	UltraHonk
Runtime	Node.js v24.0.1	Node.js v24.0.1
Trusted Setup	Hermez ptau-22	N/A

G.3 Repository and Commands

Table 27: Benchmark reproduction commands.

Step	Command
Circom	
Repository	<code>https://github.com/---anonymized---/---</code>
Install	<code>cd scripts/benchmark && npm install</code>
Generate emails	<code>npx tsx src/generate-keys.ts && npx tsx src/generate-emails.ts</code>
Count constraints	<code>npx tsx src/count-constraints.ts</code>
Generate inputs	<code>npx tsx src/generate-inputs.ts</code>
Run proving	<code>npx tsx src/run-proving.ts</code>
Noir	
Repository	<code>https://github.com/---anonymized---/-----</code>
Install	<code>cd scripts/benchmark && npm install</code>
Generate circuits	<code>npx tsx src/generate-circuits.ts</code>
Count gates	<code>npx tsx src/count-gates.ts</code>
Generate inputs	<code>npx tsx src/generate-inputs.ts</code>
Run proving	<code>npx tsx src/run-proving.ts</code>

G.4 Data File Locations

Table 28: Benchmark data files (relative to repository root).

Data	Path
Circom constraints + proving	<code>scripts/benchmark/results/raw/benchmark_2026-02-18*.json</code>
Circom summary	<code>scripts/benchmark/results/summary_2026-02-18*.md</code>
Circom configs	<code>scripts/benchmark/config/circuits.config.ts</code>
Noir gates	<code>scripts/benchmark/results/gates_2026-02-17*.json</code>
Noir proving	<code>scripts/benchmark/results/proving_2026-02-17*.json</code>
Noir CSV summary	<code>scripts/benchmark/results/csv/benchmark_2026-02-17*.csv</code>
Noir configs	<code>scripts/benchmark/config/circuits.config.ts</code>